Caio Xavier da Silva

Análise Estática Generativa com LLMs: Otimizando a Detecção de Vulnerabilidades no SemGrep via Abordagem Multi-Agente

São Paulo
2026

Caio Xavier da Silva

Análise Estática Generativa com LLMs: Otimizando a Detecção de Vulnerabilidades no SemGrep via Abordagem Multi-Agente

Trabalho de Conclusão de Curso II apresentado ao SENAC Santo Amaro como requisito parcial para conclusão do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Eduardo Heredia xxxxxx
Coorientador: Thyago Conchado Quintas

São Paulo
2026

Resumo

As ferramentas de análise estática são fundamentais na detecção precoce de vulnerabilidades de *software*, porém as abordagens tradicionais enfrentam limitações devido à dependência de lógicas rígidas criadas manualmente, o que prejudica a escalabilidade e a detecção de *corner-cases*. Em contrapartida, a aplicação direta de Modelos de Linguagem de Larga Escala (LLMs) para o escaneamento integral de bases de código acarreta custos computacionais proibitivos e riscos de alucinação. Diante deste cenário, o objetivo deste trabalho é desenvolver e avaliar um *framework* multi-agente baseado em LLMs, capaz de sintetizar e otimizar automaticamente regras de análise estática para a ferramenta SemGrep, com foco em vulnerabilidades críticas (CWEs) da linguagem C. A pesquisa adota a metodologia *Design Science Research* (DSR) e propõe um ciclo de quatro fases: construção de um *dataset* baseado no Juliet Test Suite e históricos de *commits* reais, geração de regras por agentes LLM, avaliação rigorosa de métricas (Precisão, *Recall* e F1-Score) e aplicação prática em projetos de código aberto. A hipótese central é que este modelo automatizado apresente taxas de detecção estatisticamente superiores às abordagens tradicionais, reduzindo a necessidade de engenharia manual e mitigando os altos custos da inferência direta por inteligência artificial.

Palavras-chave: análise estática. semgrep. LLM. regras. C.

Abstract

Keywords:

Lista de Figuras

Figura 1 – Motivação para *checkers* feitos por llms. 13
Figura 2 – Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python. 17
Figura 3 – *Query* utilizada durante a pesquisa de materiais. 18
Figura 4 – Funil de pesquisa. 19
Figura 5 – Árvore Sintática Abstrata para a expressão 9-5+2 21
Figura 6 – Exemplo de regra simplificada para detecção de falhas OSCI em Java. . 25
Figura 7 – *Overview* do KNighter 41
Figura 8 – Análise de CWEs (*Memory Leak*, *Use After Free*, *Null Pointer Dereference*) 47
Figura 9 – Arquitetura do pipeline proposto 53
Figura 10 – *Pipeline* para gerador de regras. 54
Figura 11 – Pós-processamento para otimizar regras e abranger *corner-cases* 54
Figura 12 – Cronograma de atividades – Diagrama de Gantt 55

Lista de Tabelas

Tabela 1 – Limitações da análise estática tradicional 11
Tabela 2 – Comparação técnica entre os trabalhos relacionados 51

Lista de Abreviaturas e Siglas

A lista a seguir reúne, em ordem alfabética, as abreviaturas e siglas utilizadas neste trabalho, acompanhadas de seus respectivos significados.

Abreviatura	**Significado**
AST	*Abstract Syntax Tree* (Árvore de Sintaxe Abstrata)
APR	*Application Programming Interface* (Reparo de Programa Automatizado)
BNF	*Backus-Naur Form* (Forma de Backus-Naur)
CFG	*Control Flow Graphs* (Grafos de Fluxo de Controle)
CNN	*Convolucional Neural Networks* (Redes Neurais Convolucionais)
CSA	*Clang Static Analyser*
CVE	*Common Vulnerabilities and Exposures* (Enumeração Comum de Vulnerabilidades e Exposições)
CWE	*Common Weakness Enumeration* (Enumeração de Fraquezas Comuns)
DFA	*Data Flow Analysis* (Análise de Fluxo de Dados)
DoS	*Denial of Service* (Negação de Serviço)
DSR	*Design Science Research*
FIA	*Flow-Insensitive Analysis* (Análise Insensível ao Fluxo)
FN	*False Negative* (Falsos Negativos)
FP	*False Positive* (Falsos Positivos)

Abreviatura	**Significado**
FSA	*Flow-Sensitive Analysis* (Análise Sensível ao Fluxo)
CFG	*Context-Free Grammar* (Gramática Livre de Contexto)
LLM	*Large Language Models* (Modelos de Larga Escala)
NLG	*Natural Language Generation* (Geração de Linguagem Natural)
NLP	*Natural Language Processing* (Processamento de Linguagem Natural)
NLU	*Natural Language Understanding* (Compreensão de Linguagem Natural)
OSCI	*Operating System Command Injection* (Injeção de Comandos no Sistema Operacional)
RL	*Reinforcement Learning* (Aprendizado por Reforço)
RNN	*Recurrent Neural Network* (Redes Neurais Recorrentes)
UAF	*Use After Free* (Uso Após Liberação)
SAST	*Static Application Security Testing*
SFDR	*Sampled False Discovery Rate* (Taxa de Falsas Descobertas Amostradas)
TN	*True Negative* (Verdadeiros Negativos)
TP	*True Positive* (Verdadeiros Positivos)

Sumário

1 INTRODUÇÃO 10
1.1 Contextualização do problema 12
1.2 Formulação do problema 15
1.3 Objetivos e hipótese 15
1.3.1 Objetivo geral 15
1.3.2 Objetivos específicos 15
1.3.3 Hipótese ou critério de validação 15
1.4 Justificativa 16

2 REVISÃO BIBLIOGRÁFICA / TRABALHOS RELACIONADOS . . 18
2.1 Metodologia da pesquisa bibliográfica 18
2.2 Fundamentos teóricos 20
2.2.1 Análise Estática Tradicional 20
2.2.1.1 Árvore Sintática Abstrata 20
2.2.1.2 Grafos de Fluxo de Controle 21
2.2.1.3 Análise de Fluxo de Dados 22
2.2.1.4 Sensibilidade de Fluxo 23
2.2.1.5 Limitações da Análise Estática 24
2.2.1.6 Análise estática baseada em regras 24
2.2.2 Vulnerabilidades de Linguagem 26
2.2.2.1 *Common Vulnerabilities and Exposures* 27
2.2.2.2 *Common Weakness Enumeration* 27
2.2.2.3 CWE-401 28
2.2.2.4 CWE-415 29
2.2.2.5 CWE-416 29
2.2.2.6 CWE-457 30
2.2.2.7 CWE-476 30
2.2.3 Linguagem Natural 31
2.2.3.1 Processamento de Linguagem Natural 32
2.2.4 Arquitetura *Transformer* 33
2.2.4.1 Modelos de Linguagens de Grande Escala 35
2.2.4.2 Agentes e Sistemas Multi-Agentes 36
2.2.5 Métricas de Avaliação 37
2.2.5.1 Juliet Test Suite 37
2.2.5.2 Validade Gramatical 37

2.2.5.3 Validade Funcional . . . 37
2.2.5.4 Validação em *Dataset* . . . 38
2.2.5.5 Validação no mundo real . . . 39
2.2.5.6 Custo . . . 39
2.3 Trabalhos correlatos . . . 39
2.3.1 Trabalho 1: *KNighter: Transforming Static Analysis with LLM-Synthesized Checkers* (YANG et al., 2025) . . . 40
2.3.2 Trabalho 2: *Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models* (HOU et al., 2025) 42
2.3.3 Trabalho 3: *RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing* (GUO et al., 2025) . . . 47
2.3.4 Síntese comparativa . . . 50

3 DESENVOLVIMENTO / IMPLEMENTAÇÃO / EXPERIMENTOS 52
3.1 Metodologia da pesquisa . . . 52
3.2 Detalhes da implementação . . . 52
3.2.1 Visão geral da solução proposta . . . 52
3.2.2 Arquitetura do trabalho . . . 53
3.3 Experimentos e cenário de testes . . . 55
3.4 Resultados e análises . . . 55
3.5 Cronograma de trabalho . . . 55

4 CONCLUSÃO . . . 56
4.1 Retomada do problema e dos objetivos . . . 56
4.2 Contribuições para a Computação . . . 56
4.3 Limitações e trabalhos futuros . . . 56

REFERÊNCIAS BIBLIOGRÁFICAS . . . 57

1 Introdução

Para dimensionar o problema tratado neste trabalho, basta observar o que acontece quando uma falha de *software* passa despercebida. Em 2014, a ausência de uma única verificação de tamanho em uma rotina de manipulação de memória da biblioteca OpenSSL, escrita em C e utilizada para proteger a comunicação de grande parte dos *sites* da internet, deu origem à vulnerabilidade conhecida como *Heartbleed* (CVE-2014-0160). A falha permitia que qualquer pessoa lesse trechos da memória dos servidores afetados, expondo senhas, dados pessoais e chaves criptográficas, e atingiu uma parcela expressiva dos servidores HTTPS mais acessados do mundo (DURUMERIC et al., 2014).

Esse caso não é isolado. A Microsoft relata que cerca de 70% das vulnerabilidades corrigidas anualmente em seus produtos são problemas de segurança de memória (Microsoft Security Response Center, 2019), e a equipe do navegador Chromium, do Google, aponta proporção semelhante entre as suas falhas de alta severidade, sendo metade delas do tipo uso de memória após a sua liberação (The Chromium Projects, 2020). Em linguagens como C, o programador é responsável por reservar e devolver manualmente a memória utilizada. Esquecer de liberá-la (vazamento de memória), liberá-la duas vezes ou continuar usando um endereço já liberado são erros simples de cometer e difíceis de perceber, mas que podem travar sistemas inteiros ou permitir que um invasor execute código malicioso. Essas falhas, catalogadas e detalhadas na Seção 2.2.2.2, constituem o foco deste trabalho.

Ferramentas de análise estática são amplamente empregadas no desenvolvimento de *software* e garantia de código (HOU et al., 2025). Por essa abordagem não necessitar de compilação e execução prévia do sistema, ela é considerada um método rápido e econômico para a identificação de problemas em seus estágios iniciais. Segundo Jiang et al. (2025), as ferramentas de análise estática emergiram como componentes indispensáveis nos fluxos de trabalhos, fornecendo mecanismos automatizados para detecção de defeitos, vulnerabilidades de segurança e violações de padrões de codificação.

~~A análise estática possui diversas vertentes. Uma estrutura fundamental conhecida é a Árvore de Sintaxe Abstrata, em inglês *Abstract* Syntax Tree (AST), uma representação intermediária que modela a estrutura hierárquica do código-fonte. ASTs fornecem uma representação de programa geralmente utilizada para análises insensíveis ao fluxo (*flow-insensitive*), onde a ordem sequencial das instruções não é estritamente necessária (MØLLER; SCHWARTZBACH, 2024).~~

~~Complementar a isso, utilizam-se os grafos de fluxo de controle, em inglês *Control* Flow Graphs (CFG). Enquanto a AST foca na sintaxe, o CFG é construído para modelar os possíveis caminhos de execução do programa, sendo essencial para análises sensíveis ao fluxo (*flow-sensitive*).~~

~~CFGs são frequentemente utilizados para a análise clássica de fluxo de dados. Nesse~~

~~processo, atribui-se a cada nó do grafo uma variável cujas restrições de fluxo (equações ou inequações) relacionam o valor do nó com o de seus vizinhos (predecessores ou sucessores) através de funções monótonas. A propagação das informações ao longo dos nós ocorre até que o algoritmo atinja o menor ponto fixo (*least* fixed point, denotado como $lfp(f)$) dentro de um reticulado completo (*complete* lattice). Essa solução matemática é a mais precisa possível para o sistema de restrições gerado e atua como uma aproximação conservadora de todas as execuções do programa, permitindo inferir propriedades seguras sobre o código analisado (MØLLER; SCHWARTZBACH, 2024).~~

Em termos simples, essas ferramentas funcionam como um revisor automático que lê o código em busca de padrões perigosos, sem precisar executá-lo. Os detalhes de como essa leitura é feita, por meio de estruturas como árvores de sintaxe e grafos de fluxo, são apresentados na Seção 2.2.1.

Tabela 1 – Limitações da análise estática tradicional

Limitação	**Descrição**
Escalabilidade e Diversidade de *Bugs*	A dependência de verificações pré-definidas ou modelagens formais restringe as ferramentas a um subconjunto limitado de padrões de *bugs*. Isso prejudica a escalabilidade e a capacidade de detectar vulnerabilidades imprevistas ou casos extremos (*corner-cases*) (YANG et al., 2025).
Esforço Humano	A configuração de verificações personalizadas exige um alto nível de especialização técnica. A complexidade sintática torna a tarefa inacessível para desenvolvedores comuns, sendo inviável mesmo com treinamento de curto prazo para realizar uma heurística eficiente (HOU et al., 2025). Pesquisas revelam que menos de 5% das lógicas de detecção utilizadas na indústria são criadas por usuários comuns (HOU et al., 2025).
Falta de Extensibilidade Multilinguagem	A maioria dos analisadores é desenvolvida com lógica rigidamente codificada (*hard-coded*) para uma linguagem de programação específica. A necessidade de construir e manter *parsers* específicos para cada nova linguagem exige alta especialização técnica, o que inviabiliza a portabilidade e gera altos custos na adaptação de analisadores para ecossistemas poliglotas (HOU et al., 2025).
Falsos Positivos e Falsos Negativos	Devido à complexidade dos sistemas de *software* e à riqueza de gramáticas de programação, é inviável que verificações manuais estáticas prevejam todas as variações estruturais. Essa generalização falha resulta em uma abundância de alarmes imprecisos no mundo real (JIANG et al., 2025).

Fonte: Elaborado pelo autor (2026)

~~Apesar da eficácia das análises baseadas em AST e CFG,~~ Apesar de sua eficácia, a implementação tradicionaldessas ferramentas frequentemente depende de heurísticas pré-

definidas, modelagens formais ou conjuntos de regras criadas manualmente para identificar padrões de falhas. Uma vez que as vulnerabilidades são detectadas, o próximo desafio no ciclo de desenvolvimento é a sua correção.

Em resposta às limitações da detecção de falhas, o campo de Reparo de Programa Automatizado, em inglês *Automated Program Repair* (APR), consolidou-se como uma área de pesquisa na engenharia de *software*. Enquanto a análise estática tradicional se limita a apontar as violações, o APR propõe-se a dar um passo além: gerar automaticamente soluções e aplicar correções de código (*patches*) sem intervenção humana, visando reduzir os custos de depuração (TANG et al., 2021).

Contudo, apesar de sua utilidade, tanto as ferramentas clássicas de detecção quanto os métodos tradicionais de correção enfrentam graves desafios estruturais. Conforme apontado por Hou et al. (2025) e Tang et al. (2021), a eficácia destas abordagens é frequentemente limitada por sua dependência de lógica rígida, o que compromete severamente a sua extensibilidade. A Tabela 1 sintetiza as principais limitações destas abordagens clássicas apontadas na literatura.

1.1 Contextualização do problema

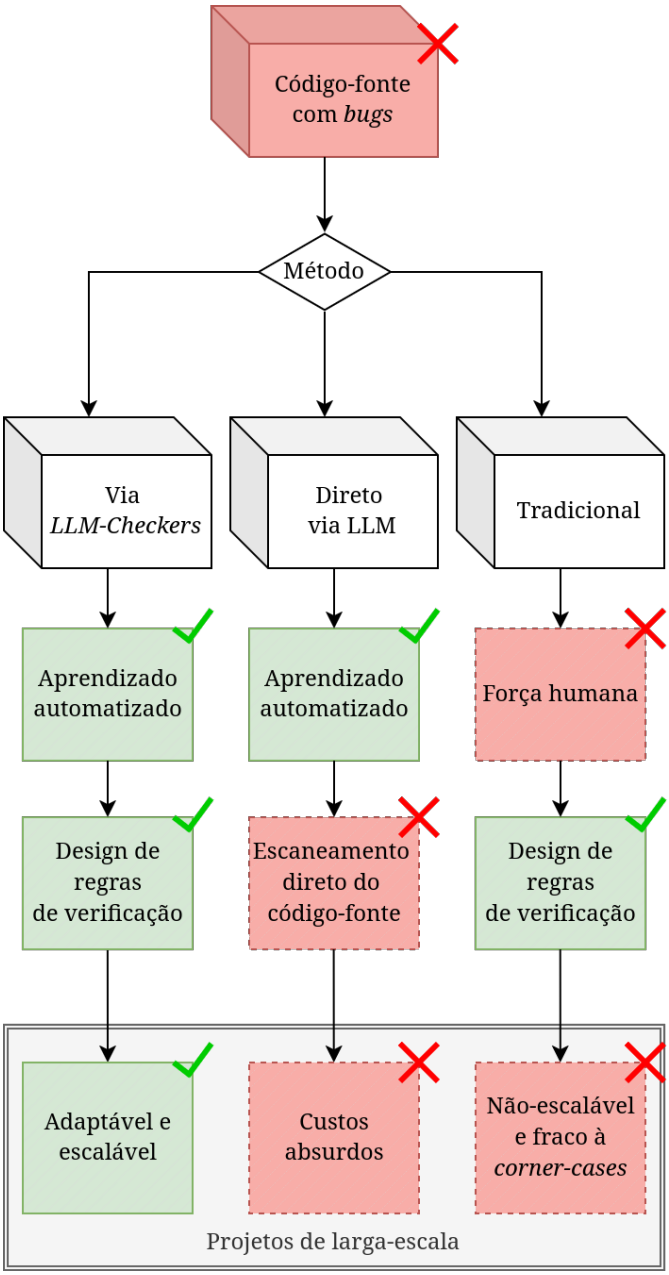
Um alarme falso, ou falso positivo, ocorre quando a ferramenta aponta como defeituoso um trecho de código que está correto; já um falso negativo é uma falha real que a ferramenta não consegue detectar. Ambos são prejudiciais: se a maior parte dos avisos for falsa, os desenvolvedores deixam de confiar na ferramenta e passam a ignorá-la (AHO et al., 2007), enquanto os falsos negativos permitem que vulnerabilidades cheguem ao usuário final.

Nos últimos anos, muitas técnicas foram desenvolvidas para minimizar alarmes falsos em analisadores estáticos (JIANG et al., 2025). Recentemente, Modelos de Linguagem de Grande Porte (LLMs) e sua ampla aplicação mudaram profundamente o domínio de análise estática, a maioria dessas técnicas empregando LLMs para compreender o código-fonte diretamente ou os dados das próprias ferramentas de análise estática, para determinar a presença de *bugs*, como sua localização e tipo (HOU et al., 2025).

Modelos de Linguagem de Larga Escala possuem o conhecimento prévio adquirido durante seu pré-treinamento em vastos repositórios de código aberto. Durante a inferência, essas capacidades podem ser direcionadas via engenharia de *prompt* (como o fornecimento de históricos de *commits* de correção no contexto) para extrair padrões sintáticos e semânticos de *bugs* específicos do sistema (TANG et al., 2021). A capacidade de analisar vários conteúdos sugere que LLMs são capazes de se adaptar a novos tipos de falhas sem a necessidade de criação explícita de verificações específicas (YANG et al., 2025).

Embora os modelos de linguagem desenvolvidos até o ano de 2026 possuam janelas de contexto na ordem de milhões de *tokens*, a inspeção direta e contínua do código-fonte esbarra em severas restrições práticas e custos proibitivos. A raiz dessa limitação reside

Figura 1 – Motivação para *checkers* feitos por llms.



Fonte: Adaptado pelo autor para melhor compreensão (YANG et al., 2025)

na arquitetura Transformer, cujo mecanismo de autoatenção (*self-attention*) possui uma complexidade computacional e de memória quadrática ($O(n^2)$) em relação ao tamanho da sequência de entrada (VASWANI et al., 2017). Como consequência, a submissão integral de bases de código de projetos em larga escala, como o *kernel* do Linux, provoca uma explosão nos requisitos de processamento (TUNSTALL; WERRA; WOLF, 2022).

Estudos empíricos confirmam que essa abordagem pode consumir dezenas de milhões de *tokens* e exigir dias de processamento contínuo, criando uma limitação crítica para a escalabilidade das ferramentas (HOU et al., 2025). Essa sobrecarga de contexto exacerba o risco de alucinações da inteligência artificial, induzindo o modelo a gerar inferências

estruturalmente plausíveis, porém tecnicamente incorretas.

Para contornar as limitações de memória e o alto custo da inferência direta por um único modelo, a engenharia de *software* tem explorado a aplicação de Sistemas Multi-Agentes. Um agente inteligente é fundamentalmente uma entidade computacional que percebe o seu ambiente e age sobre ele de alguma forma (RUSSELL; NORVIG, 2020). Em vez de submeter bases de código gigantescas a uma única instância de inteligência artificial, o processamento é descentralizado e distribuído entre múltiplos agentes.

Em um ambiente cooperativo, a comunicação e a divisão de tarefas entre os agentes emergem como comportamentos racionais essenciais para a resolução de problemas complexos (RUSSELL; NORVIG, 2020). Dessa forma, cada agente LLM assume um papel restrito e especializado, como abstração, síntese de sintaxe ou validação, colaborando de forma iterativa para reduzir a sobrecarga de *tokens* e aumentar a precisão na geração do artefato final.

A figura 1 introduz motivações para a necessidade de métodos mais eficientes e, principalmente, especializados, tal que em projetos grandes, a análise estática possui dois desafios: criar regras para diversos padrões de bugs e gerenciamento de bases de códigos imensas (YANG et al., 2025).

Considerando esses dados, o analisador estático ideal deveria: detectar uma diversidade de falhas, incluindo *corner-cases* e semânticas específicas de sistemas, e eficientemente processar milhões de linhas de código (YANG et al., 2025). Porém, dos métodos existentes, geralmente sacrificam um desses objetivos (YANG et al., 2025).

O método apresentado por Yang et al. (2025), KNighter, é um analisador estático sintetizado utilizando LLMs, que ao invés de métodos diretos de utilização de LLMs, é realizado um treinamento de padrões de *bugs* a partir de histórico de *patches* que a partir disso é dedicado para um *checker* especializado. A partir desse paradigma, Yang et al. (2025) sobrepassa os problemas de limites de contextos e custos absurdos de analisar bases de códigos de milhões de linhas com diversos bugs.

Uma alternativa ao uso de analisadores construídos do zero são as ferramentas de análise estática baseadas em regras, como o SemGrep. O SemGrep é uma ferramenta de código aberto na qual o usuário descreve, em arquivos no formato YAML, regras cujos padrões imitam a própria sintaxe da linguagem analisada, como ilustra a Figura 2 (HOU et al., 2025). Por dispensar a construção de um analisador específico para cada tipo de falha, Hou et al. (2025) utilizam o SemGrep como base para o conjunto de regras (*ruleset*) que geram por meio de LLMs, abordagem também adotada neste trabalho. Seu funcionamento é detalhado na Seção 2.2.1.6.

1.2 Formulação do problema

1.3 Objetivos e hipótese

Os objetivos desta pesquisa estruturam-se em duas frentes: o objetivo geral, focado em solucionar o problema central do estudo, e os objetivos específicos, representam os subprodutos ou detalhamentos lógicos gerados para alcançar esse propósito maior.

1.3.1 Objetivo geral

O objetivo geral deste trabalho é demonstrar que um *framework* de síntese automatizada de regras, baseando-se em Modelos de Linguagem de Larga Escala e integrado à ferramenta SemGrep, apresenta precisão e *recall* estatisticamente superiores às abordagens estáticas tradicionais na detecção de vulnerabilidades críticas em linguagem C. Busca-se demonstrar que este modelo supera abordagens estáticas tradicionais na identificação precisa de, no mínimo, cinco categorias de vulnerabilidades de software, em inglês *Common Weakness Enumeration* (CWE).

1.3.2 Objetivos específicos

Sobre os objetivos específicos, temos:

1. Determinar os limites de detecção semântica (*corner-cases*) que podem ser superados pela abordagem gerativa de LLMs em relação aos conjuntos de regras predefinidas (*ruleset*) estáticos.

2. ~~Estabelecer~~ Avaliar o ganho quantitativode desempenho (*Recall*, Precisão e F1-Score) de regras sintetizadas dinamicamente frente aos analisadores tradicionais do estado da arte.

3. Determinar a viabilidade técnica e a escalabilidade de inferência da arquitetura proposta utilizando modelos de linguagem de menor escala (com contagem reduzida de parâmetros), visando garantir a viabilidade técnica da execução do *framework* em infraestruturas locais com restrições de custo e *hardware*.

1.3.3 Hipótese ou critério de validação

A hipótese desta pesquisa é a de que a geração automatizada de regras por meio de um *framework* multi-agente baseado em LLMs, integrado ao motor do SemGrep, produzirá taxas de detecção (*Recall* e *F1-Score*) de vulnerabilidades (CWEs) em C estatisticamente superiores e com maior identificação de *corner-cases* quando comparada aos conjuntos

de regras predefinidas dos analisadores estáticos tradicionais (como por exemplo *ruleset* tradicional do SemGrep).

1.4 Justificativa

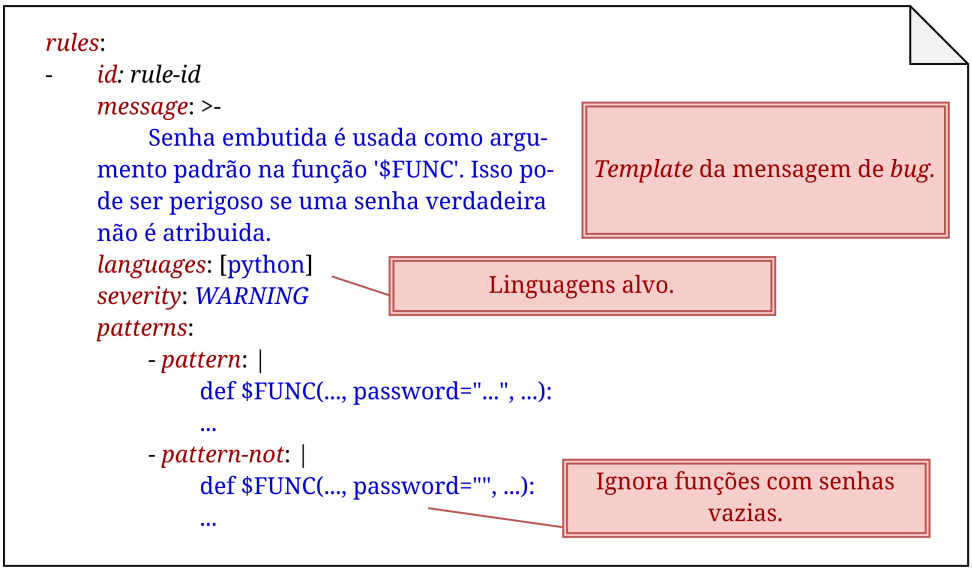
A literatura recente demonstra avanços significativos na aplicação de Modelos de Linguagem de Larga Escala, em inglês *Large Language Models* (LLM), para a análise estática, comprovando a viabilidade de sintetizar verificadores automatizados de código (HOU et al., 2025; YANG et al., 2025). Trabalhos recentes atestam que a integração de LLMs com ferramentas ágeis e baseadas em regras, como o SemGrep, supera as abordagens tradicionais de lógica rígida, permitindo uma maior flexibilidade e abrangência na identificação de vulnerabilidades (HOU et al., 2025; YANG et al., 2025; LI et al., 2026).

Contudo, apesar das evoluções trazidas pelo estudo pioneiro de Yang et al. (2025), estudos publicados em 2025 e 2026 revelam que métodos baseados em LLMs ainda exibem uma cobertura limitada, frequentemente falhando em lidar com variações semânticas complexas e *corner-cases* (HOU et al., 2025; YANG et al., 2025). Porém estudos concluíram que métodos de análise estática baseado em LLMs identificam mais vulnerabilidades únicas do que métodos tradicionais (YANG et al., 2025; HOU et al., 2025; LI et al., 2026).

~~Hou et al. (2025) citam a ferramenta *Semgrep*, uma ferramenta utilizada para montagem de regras personalizadas para análise estática, como na Figura 2, tal que a utilizam para o *ruleset* que geram via LLMs.~~ (Explicação do SemGrep movida para o fim da Seção 1.1, antes de sua primeira menção.)

Portanto, este trabalho justifica-se pela necessidade de preencher a atual lacuna de eficácia e escalabilidade na detecção de vulnerabilidades. ~~O desenvolvimento de um *framework* iterativo, focado na geração automatizada de regras para o motor do SemGrep, é motivado pela urgência em mitigar a dependência de engenharia manual e contornar os altos custos da inferência direta por IAs.~~ O desenvolvimento de um *framework* iterativo, focado na geração automatizada de regras para o motor do SemGrep, é motivado por dois problemas concretos. O primeiro é a dependência de engenharia manual: escrever regras de detecção exige conhecimento especializado, a ponto de menos de 5% das lógicas de detecção utilizadas na indústria serem criadas por usuários comuns (HOU et al., 2025). O segundo é o custo da inferência direta por IAs: submeter projetos inteiros a um LLM pode consumir dezenas de milhões de *tokens* e dias de processamento por projeto (LI et al., 2026), o que inviabiliza o seu uso contínuo. Gerar as regras uma única vez e reutilizá-las no SemGrep, cuja execução não depende de LLMs, ataca os dois problemas simultaneamente. Ao propor um sistema especializado nas vulnerabilidades críticas (CWEs) da linguagem C, essa pesquisa visa demonstrar que é possível identificar padrões semânticos e *corner-cases* com maior precisão e viabilidade técnica do que os analisadores estáticos tradicionais ou métodos generalistas baseados exclusivamente em LLMs.

Figura 2 – Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python.



Fonte: Adaptado para português com finalidade de melhorar compreensão (HOU et al., 2025).

2 Revisão Bibliográfica / Trabalhos Relacionados

Este capítulo tem como finalidade estabelecer o embasamento teórico e tecnológico que sustenta a pesquisa proposta. Inicialmente, descreve-se a metodologia utilizada para a seleção dos trabalhos consultados. Na sequência, o referencial teórico explora os paradigmas da análise estática assistida por inteligência artificial, as limitações da análise dinâmica e os desafios inerentes à análise estática tradicional. Os fundamentos técnicos são apresentados com ênfase em Modelos de Linguagem de Grande Escala (LLMs), Processamento de Linguagem Natural e os princípios de Análise Estática. Por fim, conduz-se uma revisão dos trabalhos correlatos, identificando as lacunas que esta pesquisa busca preencher.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida de forma sistemática para identificar trabalhos científicos relevantes sobre a utilização de LLMs no suporte à análise estática. As buscas foram centralizadas no motor Google Acadêmico (*Google Scholar*), devido à sua ampla indexação, com foco principal em bases de dados consolidadas na área da Computação, como arXiv, IEEE Xplore e ACM Digital Library.

Para as buscas, foram definidas palavras-chave em inglês, tais como: *"static analysis"*, *"LLM"*, *"large language model"*, *"GPT"*, *"C"*, *"C++"*, *"rule generation"*, *"Linux kernel"*, *"pointer"*, *"undefined"*, *"memory"*, *"value-flow"* e *"SemGrep"*. Estes termos foram combinados por meio de operadores booleanos (*AND*, *OR*) para refinar os resultados, culminando na *string* de busca apresentada na Figura 3:

Figura 3 – *Query* utilizada durante a pesquisa de materiais.

```
"static analysis" AND ("LLM" OR "large language model" OR "GPT") AND ("C" OR "C++" OR
"Linux kernel") AND ("pointer" OR "undefined" OR "memory" OR "value-flow") AND
("SemGrep" OR "semgrep")
```

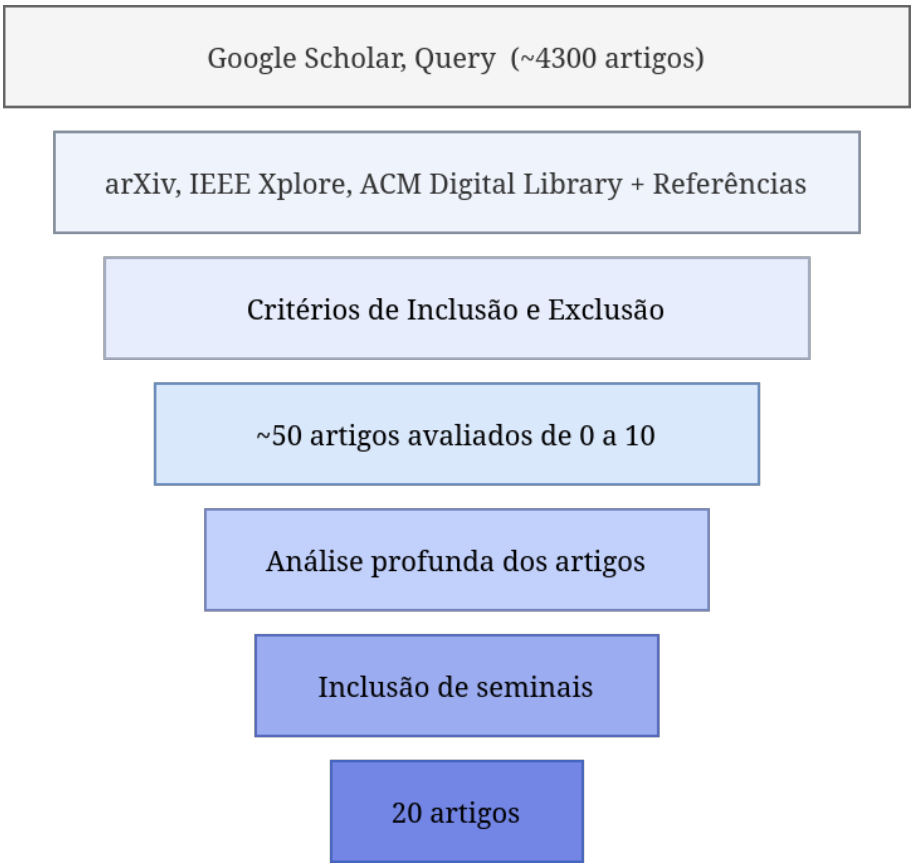
Fonte: Elaborado pelo autor (2026)

Como critérios de inclusão, estabeleceram-se: (a) trabalhos publicados entre 2021 e 2026, de forma a capturar o estado da arte; (b) disponibilidade nos idiomas inglês ou português; e (c) relação direta com o tema, priorizando artigos que contivessem implementações ou experimentos aplicados à utilização de LLMs para geração de regras para a linguagem C.

Em contrapartida, os critérios de exclusão adotados foram: (a) trabalhos publicados antes de 2021, com exceção de referências clássicas ou seminais indispensáveis para a teoria; (b) artigos sem alinhamento direto com análise estática ou geração de regras de código; e (c) duplicidades de publicações entre as bases de dados consultadas.

O processo de seleção ocorreu em duas etapas. Na primeira fase (triagem), a aplicação da *string* de busca retornou 4.770 resultados, dos quais foram selecionados 47 artigos potencialmente relevantes após a leitura prévia de títulos e resumos. Na segunda fase (afunilamento), os trabalhos restantes passaram por uma leitura criteriosa da introdução e do escopo, sendo classificados em uma escala de relevância de 0 a 10 em relação aos objetivos desta pesquisa.

Figura 4 – Funil de pesquisa.



Fonte: Elaborado pelo autor (2026)

A avaliação de 0 a 10 foi feita a partir de seu resumo, palavras-chave, tecnologia aplicada. Caso o trabalho possua similaridades com outros artigos bem avaliados é considerado como suporte ou alta relevância. Esse processo de filtragem resultou em 6 artigos classificados com notas entre 9 e 10 (alta relevância), além de 9 artigos categorizados como material de suporte, totalizando 15 trabalhos contemporâneos para compor a fundamentação teórica. Adicionalmente, foram incorporados 5 trabalhos seminais anteriores a 2021, essenciais para a consolidação dos conceitos clássicos abordados neste estudo.

2.2 Fundamentos teóricos

O referencial teórico apresenta os conceitos fundamentais que sustentam o desenvolvimento deste projeto. Propõe-se um contraponto analítico entre os métodos da análise estática tradicional e as novas abordagens voltadas à automatização da geração de regras. A estruturação dos tópicos foi concebida para apresentar gradativamente as tecnologias envolvidas, destacando o papel das LLMs e sua relevância para a modernização das ferramentas de verificação de código na atualidade.

2.2.1 Análise Estática Tradicional

Verificações estáticas, segundo Aho et al. (2007), são checagens de consistência realizadas durante o processo de compilação, sendo possível capturar erros de programação cedo, antes da sua execução.

Incluindo dois tipos de análises: **(i) Verificação Sintática**, envolve a aplicação de restrições que não são aplicadas apenas pelas gramáticas, como um identificador ser declarado apenas uma vez dentro do escopo (AHO et al., 2007). **(ii) Verificação de Tipos**, assegura que as regras de tipagem da respectiva linguagem sejam seguidas, checando se os operadores ou funções estão sendo aplicados ao número ou tipo correto de operando. Se a conversão de tipos é necessária, o verificador pode inserir o operador correto na árvore de sintaxe para a conversão (AHO et al., 2007).

Não é importante somente para otimização de código, as mesmas técnicas podem ser usadas para analisar *software* já existentes para diversos padrões comuns de erros de programação (AHO et al., 2007). PREfix e Metal são duas ferramentas mencionadas por Aho et al. (2007) utilizam análise estática interprocedural, rastreando o fluxo de informações em um programa inteiro, para encontrar erros de programação estaticamente em grandes bases de código. Estas ferramentas são capazes de rastrear o caminho de entradas fornecidas pelos usuários enquanto essas *strings* são copiadas ou combinadas ao longo de vários procedimentos, acompanhando as entradas é possível prevenir contra *SQL Injection* e *Buffer Overflows* (AHO et al., 2007).

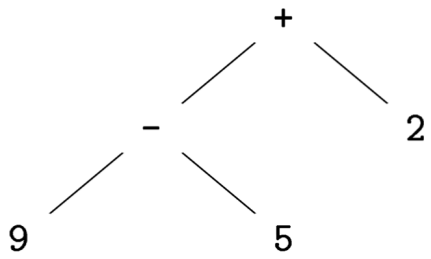
2.2.1.1 Árvore Sintática Abstrata

Durante o processo de análise sintática, é comum a geração de uma árvore de análise (*Parse Tree*), também conhecida como árvore de sintaxe concreta, onde os nós internos representam os símbolos não-terminais definidos pela gramática da linguagem (AHO et al., 2007). Contudo, para as fases subsequentes de compilação ou de análise estática de código, essa representação contém detalhes puramente estruturais que não afetam a semântica do programa (AHO et al., 2007).

A Árvore Sintática Abstrata, do inglês *Abstract Syntax Tree* (AST), surge como uma representação intermediária otimizada. Em uma AST, cada nó interno representa

um operador ou um constructo de programação, enquanto nós filhos representam os operandos ou os componentes semânticos daquele constructo (AHO et al., 2007). Nós auxiliares exigidos pela gramática para contornar ambiguidades ou definir precedência (como produções única ou parênteses) são descartados, resultando em uma estrutura de dados mais limpa e focada no significado das operações (AHO et al., 2007).

Figura 5 – Árvore Sintática Abstrata para a expressão 9-5+2



Fonte: (AHO et al., 2007)

Tomando como exemplo, na figura 5, a expressão matemática 9 - 5 + 2, a raiz da AST representaria o operador +, tendo como filho esquerdo a subexpressão correspondente a 9 - 5 e como filho direito a constante 2. Esse agrupamento estrutural reflete a precedência e a associatividade à esquerda dos operadores, tornando redundante a presença de nós sintáticos adicionais.

Em ferramentas de análise estática voltadas para linguagens complexas como C, operar sobre a AST é essencial, pois permite que a verificação de propriedades e a busca por vulnerabilidades sejam conduzidas diretamente sobre a lógica estrutural do código, isolando a análise de variações gramaticais irrelevantes (AHO et al., 2007).

No contexto da análise de vulnerabilidades, a AST é fundamental para identificar padrões estruturais perigosos. Ao invés de realizar buscas textuais limitadas, os analisadores percorrem a árvore em busca de composições específicas de nós.

Por exemplo, para detectar um risco de desreferenciação de ponteiro nulo (*Null Pointer Dereference*), uma regra de análise pode inspecionar a AST em busca de um nó que represente uma chamada de alocação de memória, como a função `devm_kzalloc`, verificando se não existe um nó condicional adjacente encarregado de checar o valor de retorno antes de sua utilização.

2.2.1.2 Grafos de Fluxo de Controle

Complementar à Árvore Sintática Abstrata (AST), utilizam-se os grafos de fluxo de controle, do inglês *Control Flow Graphs* (CFG). Enquanto a AST foca em representar hierarquicamente a sintaxe estrutural do código, o CFG é um grafo direcionado construído

para modelar os possíveis caminhos de execução do programa, sendo a base essencial para análises sensíveis ao fluxo (*flow-sensitive*) (AHO et al., 2007).

Representação em grafo de código intermediário é muito útil para a geração de código (AHO et al., 2007). De acordo com Aho et al. (2007), essa representação é construída em dois passos, primeiro, é particionado o código intermediário em blocos básicos (*basic blocks*), que atuam como os nós do grafo.

Um bloco básico é uma sequência máxima de instruções consecutivas de três endereços onde o fluxo de controle entra obrigatoriamente pela primeira instrução, sem saltos para o interior dele, e sai apenas pela última, sem interrupções ou ramificações em seu interior (AHO et al., 2007). As arestas relacionadas do grafo indicam as transições de controle (*control flow*) permitidas no programa, representando que a execução pode fluir do final de um bloco básico específico para o início de outro (AHO et al., 2007).

Na construção de um CFG, as instruções de desvio (*jumps*) endereçadas a linhas ou rótulos específicos são substituídas por referências diretas aos blocos básicos correspondentes. Essa abstração é fundamental na construção de compiladores, pois permite que otimizações subsequentes alterem livremente o conteúdo interno dos blocos sem invalidar os alvos dos desvios de controle (AHO et al., 2007). Caso não houvesse essa otimização, seria necessário corrigir os alvos toda vez que uma instrução fosse alterada.

CFGs podem ser representados na programação por qualquer estrutura de dados tradicional para grafos. Quanto ao conteúdo interno dos nós (blocos básicos), Aho et al. (2007) afirma que eles podem ser representados por um ponteiro para a primeira instrução e uma contagem de instruções. No entanto, como as otimizações alteram frequentemente a quantidade de instruções, costuma ser mais eficiente representar o conteúdo de cada bloco básico, ou nó, como uma lista encadeada (*linked list*) de instruções (AHO et al., 2007).

2.2.1.3 Análise de Fluxo de Dados

A Análise de Fluxo de Dados, em inglês *Data Flow Analysis* (DFA), é definida como um conjunto de técnicas que derivam informações sobre o fluxo de dados ao longo dos possíveis caminhos de execução de um programa (AHO et al., 2007).

Tradicionalmente fundamentais para as otimizações globais em compiladores, essas técnicas permitem determinar, por exemplo, se duas expressões avaliam para o mesmo valor em todos os caminhos possíveis ou se o resultado de uma atribuição nunca é utilizado (código morto) (AHO et al., 2007).

A execução de um programa é modelada como uma série de transformações do estado do programa, onde o estado consiste nos valores de todas as variáveis do programa em um dado momento (AHO et al., 2007). Cada instrução executada transforma um estado de entrada em um novo estado de saída.

No entanto, Aho et al. (2007) afirma que é impossível acompanhar todos os estados exatos devido ao número ilimitado de caminhos de execução. Por esse motivo, a DFA opera

através de abstrações, ignorando detalhes irrelevantes e guardando apenas as características essenciais para o objetivo da otimização (AHO et al., 2007).

É associado a cada ponto do programa um valor de fluxo de dados (que representa a abstrações dos estados possíveis) pertencente a um "domínio" (AHO et al., 2007). O problema de DFAs consiste em encontrar uma solução para dois conjuntos de restrições principais:

Funções de transferência (*Transfer Functions*), são restrições baseadas na semântica de cada instrução, pegando o valor do fluxo de dados antes da instrução e geram um novo valor de saída (AHO et al., 2007). O fluxo pode acontecer "para frente" (*forward*), onde a entrada molda a saída, ou "para trás" (*backward*), onde a direção desse fluxo é invertida.

Restrições de Fluxo de Controle (*Control-Flow Constraints*), definem como a informação transita de um bloco básico para outro dentro do CFG. Essencialmente, o valor do fluxo de dados que entra em uma instrução é matematicamente determinado pela combinação dos valores que saem das instruções ou blocos predecessores (AHO et al., 2007).

No contexto da segurança de *software* e da detecção de falhas abordadas neste trabalho, a abstração fornecida pela DFA é o motor principal que permite analisadores estáticos avançados identificarem vulnerabilidades complexas. É através do rastreamento do estado das variáveis pelos caminhos de controle que se torna viável detectar comportamentos críticos em linguagens como C, como *Null Pointer Dereference* (CWE-476) ou uso de variáveis não inicializadas (CWE-457).

2.2.1.4 Sensibilidade de Fluxo

A precisão de uma análise estática depende fortemente de sua sensibilidade ao fluxo de controle. Segundo Aho et al. (2007), as abordagens de análise (como a análise de apontamento de ponteiros) podem ser categorizadas em dois tipos principais:

Análise Sensível ao Fluxo, em inglês *Flow-Sensitive Analysis* (FSA), essa abordagem acompanha o fluxo de controle do programa e computa o estado exato (por exemplo, para onde cada variável pode apontar) após a execução de cada instrução individual Aho et al. (2007). Isso significa que a análise avalia as restrições e funções de transferência passo a passo; ela não apenas considera novas informações e relações que um comando gera (*generate*), mas também contabiliza as informações anteriores que o comando invalida ou mata (*kills*).

Análise Insensível ao Fluxo, em inglês *Flow-Insensitive Analysis* (FIA), em contrapartida, essa abordagem ignora o fluxo de controle, assumindo essencialmente que cada instrução do programa pode ser executada em qualquer ordem (AHO et al., 2007). A FIA computa apenas um mapa global indicando para onde cada variável possivelmente apontar em qualquer momento da execução do programa. Em uma análise insensível ao fluxo, uma atribuição nunca invalida (*kill*) relações de apontamento; ela apenas gera novas relações.

Ou seja, se uma variável puder apontar para dois objetos diferentes após a avaliação de duas instruções distintas, o sistema apenas registra e acumula a informação global de que ela aponta para ambos (AHO et al., 2007).

Aho et al. (2007) explicam que a falta de sensibilidade ao fluxo enfraquece consideravelmente a precisão dos resultados da análise, uma vez que produz superaproximações do estado real do programa. No entanto, essa abstração tem o benefício direto de reduzir drasticamente o tamanho da representação dos resultados e fazer com que o algoritmo termine de forma mais rápida, o que é frequentemente necessário ao analisar bases de código em larga escala.

2.2.1.5 Limitações da Análise Estática

Aho et al. (2007) explicam que o problema de encontrar todos os erros de um programa é indecidível. Uma ferramenta de análise de fluxo de dados pode até ser projetada para alertar programadores sobre todas as possíveis partes do código com uma categoria específica de erros, porém se a maioria desses avisos forem falsos alarmes, os usuários não vão utilizar a ferramenta.

Por conta dessa complexidade, Aho et al. (2007) afirmam que os detectores práticos de erros frequentemente não são perfeitos (à prova de falhas ou *sound*) nem completos (*complete*). As abordagens tradicionais de detecção de vulnerabilidades estática geralmente codificam o conhecimento especializado como regras projetadas manualmente, com padrões e modelos de fluxo de dados, sua dependência de padrões predefinidos limita inerentemente a generalização e a cobertura para tipos de vulnerabilidades previamente não vistos ou em evolução (LI et al., 2026).

Isso significa que a análise estática pode não encontrar todos os erros no programa, e nem todos os erros relatados têm a garantia de serem erros reais, e se as ferramentas relatassem todos os erros potenciais, o volume de falsos positivos tornaria as ferramentas inutilizáveis (AHO et al., 2007).

2.2.1.6 Análise estática baseada em regras

Em anos recentes, ferramentas de análise estática baseada em regras foram introduzidas. Complementando as ferramentas tradicionais, ao identificar previamente problemas durante o desenvolvimento pré-produção, reduzem significamente o custo e complexidade de *debugging* e manutenção (JIANG et al., 2025). Dos componentes mais utilizados, um dos mais relevantes são SemGrep e CodeQL (JIANG et al., 2025; HOU et al., 2025).

Formalmente, SemGrep é uma ferramenta de análise estática que suporta estruturas complexas e verificações básicas de fluxo de dados no código, permitindo inspeções sem execução para identificar vulnerabilidades de segurança, qualidade de código e violações a conformidade (HOU et al., 2025).

Sua gramática baseada em YAML mimetiza a sintaxe da linguagem alvo, reduzindo sua curva de aprendizado ao contrário de seu concorrente CodeQL (HOU et al., 2025), que possui sua própria gramática lógica, tratando o código como um banco de dados relacional, sendo possível realizar análises profundas porém custosa em tempo de aprendizado.

SemGrep converte código em uma árvore de sintaxe abstrata universal (UAST), aplicando um *pattern-matching* uniforme em mais de 30 linguagens (HOU et al., 2025). ~~Com sua ativa continuidade e plataforma capaz de compartilhar regras existentes melhora sua adaptabilidade e desenvolvimento colaborativo.~~ Por ser um projeto mantido ativamente e contar com uma plataforma pública na qual a comunidade compartilha regras prontas, o SemGrep facilita a adaptação de regras existentes e o desenvolvimento colaborativo.

Figura 6 – Exemplo de regra simplificada para detecção de falhas OSCI em Java.

```
1   patterns:
2       - pattern-inside: |
3           $FUNCDECL(..., $USER_DATA, ...) {...}
4       - pattern-either:
5           - pattern: |
6               $PB.command(..., $USER_DATA, ...)
7           - patterns:
8               - pattern-either:
9                   - pattern-inside: |
10                      $X = $USER_DATA; ...
11                  - pattern-inside: |
12                      $X = trim($USER_DATA); ...
13              - pattern: $PB.command(..., $X, ...);
```

Fonte: Adaptado de (JIANG et al., 2025).

~~Jiang et al. (2025) explica com a figura 6 que a partir de uma regra no SemGrep com o propósito de identificar uma falha de segurança (como a injeção de comandos no sistema operacional), do inglês *Operating* System Command Injection (OSCI), em Java, sendo possível visualizar a utilidade prática desse *framework*.~~ Jiang et al. (2025) utilizam a regra da Figura 6 para ilustrar a utilidade prática do SemGrep: ela identifica, em código Java, uma falha de injeção de comandos no sistema operacional, do inglês *Operating System Command Injection* (OSCI).

Para que a busca seja flexível e rastrear fluxo de dados, o SemGrep utiliza dois recursos fundamentais:

(i) **Metavariáveis** (ex.: *$USER_DATA*, *$X*), a partir de um cifrão e nome da variável em letras maiúsculas. Essas variáveis são atribuídas quando SemGrep identifica um padrão que se encaixa, sendo "salvo" o valor na metavariável, consequentemente, se a metavariável

é reutilizada na mesma regra, o SemGrep exigirá que o valor capturado seja exatamente o mesmo, o que permite *Data Flow*.

(ii) **Ellipsis/Curinga** (...), representando qualquer sequência de coisas. Assim independente da quantidade de argumentos em uma função, número de linhas de código entre duas instruções, é possível omitir detalhes não importantes com (...), tornando SemGrep uma ferramenta abstraída.

Os predicados são as afirmações que procuram o código. *Tags* como *pattern* ou *pattern-inside* são predicados, com ***pattern*** sendo uma afirmação exata, como na linha 5, e ***pattern-inside*** define o contexto, não apontando a vulnerabilidade exata, mas exige que qualquer correspondência subsequente aconteça dentro da estrutura dele, como na linha 2.

A partir destes predicados, é possível combinar operações lógicas (*AND*, *OR*, *NOT*) sendo, respectivamente, *patterns*, *pattern-either*, *pattern-not*.

(i) O predicado ***patterns*** representa a lógica *AND* (E), para toda regra ou condição listada abaixo deste bloco devem ser verdadeiras simultaneamente. Como na linha 1, o bloco principal exige que o código esteja dentro da função (linha 2) **E** corresponda a um dos casos de injeção (linha 4).

(ii) ***pattern-either*** representa a lógica *OR* (OU), se o código corresponde a qualquer uma das condições aninhadas sob este predicado, a avaliação será verdadeira. Na linha 4, a regra procura um fluxo direto de *$USER_DATA* ou um fluxo indireto por atribuição.

(iii) ***pattern-not*** representa a lógica *NOT* (NÃO). Não presente no exemplo, porém usado para excluir falsos positivos, anulando caso a regra bater com a condição aninhada no predicado.

~~Com estes conhecimentos, é possível compreender mais profundamente como a ferramenta utilizada funciona. SemGrep possui maneiras de lidar com falsos positivas nativamente, nos permitindo liberdade para aperfeiçoar a análise de regras.~~ Com estes conhecimentos, é possível compreender como o SemGrep lida nativamente com falsos positivos. Predicados de exclusão, como *pattern-not* e *pattern-not-inside*, descartam ocorrências que já estão protegidas, por exemplo, o uso de um ponteiro que foi verificado contra `NULL` logo antes, enquanto *pattern-inside* restringe a busca ao contexto em que a falha de fato pode ocorrer. Assim, cada regra pode ser refinada de forma incremental, acrescentando exclusões para os falsos positivos observados, o que é explorado pelo *pipeline* proposto neste trabalho.

2.2.2 Vulnerabilidades de Linguagem

O termo Vulnerabilidade (*Vulnerability*) é definido como um conjunto de condições que permite que um invasor (atacante) viole uma política de segurança explícita ou implícita (SEACORD, 2013; Software Engineering Institute, 2016). Nem toda falha de segurança (*security flaw*) leva a uma vulnerabilidade (SEACORD, 2013).

Seacord (2013) explicam que para que uma falha torne um programa vulnerável a ataques, é necessário que os dados de entrada desse programa cruzem um limite de segurança (*security boundary*). Uma falha pode existir no código sem as pré-condições para ser explorada; por exemplo, se o programa apresentar um erro grave mas não possuir permissões para ser acessado localmente, ele não viola a política de segurança e não constitui uma vulnerabilidade ativa (SEACORD, 2013).

Uma vulnerabilidade pode existir mesmo sem que haja uma falha ou defeito no código (SEACORD, 2013). Como a segurança é um atributo de qualidade que muitas vezes precisa ser balanceado com outros fatores (como desempenho e usabilidade), os projetistas de *software* podem decidir intencionalmente deixar um produto vulnerável a certas formas de exploração (SEACORD, 2013). Isso representação uma aceitação consciente do risco pelo projetista e não um erro de programação.

2.2.2.1 *Common Vulnerabilities and Exposures*

A Enumeração Comum de Vulnerabilidades e Exposições, mais popularmente conhecida do inglês *Common Vulnerabilities and Exposures* (CVE). Similarmente à CWEs, CVE atua como um dicionário, com foco em identificar, definir e catalogar vulnerabilidades de segurança cibernética que foram divulgadas publicamente (CVE Program, 2026).

CVE possui um único registro no seu catálogo para cada vulnerabilidade descoberta para uma versão específica de um *software*. O objetivo principal de se atribuir um CVE é permitir que profissionais de TI, ferramentas automatizadas e equipes de segurança cibernética saibam que estão se referindo exatamente ao mesmo problema (CVE Program, 2026).

Após a confirmação de uma vulnerabilidade, ela recebe um identificador único composto pelo ano de relato ou atribuição, seguido por um sequencial numérico. Parceiros como *National Vulnerability Database* (NVD) adicionam o registro com suas descrições, atribuindo um índice de severidade metrificado pela *Common Vulnerability Scoring System* (CVSS) e sua causa raiz proveniente da CWE (CVE Program, 2026).

2.2.2.2 *Common Weakness Enumeration*

A Enumeração de Fraquezas Comuns, mais popularmente conhecida do inglês *Common Weakness Enumeration* (CWE), é um dicionário formal e uma lista desenvolvida pela comunidade que cataloga fraquezas comuns de *software* e *hardware* (MITRE Corporation, 2026a).

Neste contexto, o termo fraqueza (*weakness*) é definido como uma condição em um componente de *software*, *firmware*, *hardware* ou serviço que, sob determinadas circunstâncias, pode contribuir para a introdução de vulnerabilidades (MITRE Corporation, 2026a). O projeto identifica, classifica e descreve esses problemas sob a nomenclatura de CWEs individuais (como, por exemplo, CWE-476 *Null Pointer Dereference*).

O propósito principal de se mapear essas fraquezas é permitir que desenvolvedores de *software*, projetistas de *hardware* e arquitetos de segurança as eliminem ainda na fase de desenvolvimento, antes da implantação (*deployment*), o que torna a correção muito mais fácil e barata.

O conteúdo de CWEs pode ser navegado através de diferentes visões, agrupando fraquezas para domínios específicos, como visões focadas estritamente em conceitos de Desenvolvimento de *Software*, organizando as falhas que surgem durante a programação), Design de *Hardware* e Conceitos de Pesquisa, organizando as falhas por comportamentos (MITRE Corporation, 2026a).

O dicionário CWE fornece mapeamentos externos essenciais e atua como base para diversos outros recursos e guias de segurança fundamentais na indústria, como as listas **CWE Top 25**, **OWASP Top 10** e os **padrões de codificação segura do SEI CERT** (MITRE Corporation, 2026a; Software Engineering Institute, 2016). A lista de fraquezas é atualizada de três a quatro vezes por ano para adicionar ou aprimorar informações, e o dicionário é construído, em parte, tomando como base os problemas reportados nos registros de vulnerabilidades ativas do catálogo CVE, do inglês *Common Vulnerabilities and Exposures*.

Para o escopo deste trabalho, serão consideradas primariamente cinco vulnerabilidades presentes na lista oficial da CWE, todas são fraquezas relacionadas a falhas de memória. São elas:

- **CWE-401**: *Missing Release of Memory after Effective Lifetime*;
- **CWE-415**: *Double Free*;
- **CWE-416**: *Use After Free*;
- **CWE-457**: *Use of Uninitialized Variable*;
- **CWE-476**: *Null Pointer Dereference*.

2.2.2.3 CWE-401

A CWE-401, chamada de Vazamento de Memória, do inglês *Missing Release of Memory after Effective Lifetime* ou *Memory Leak*, ocorre quando um programa aloca memória, mas falha em rastreá-la ou liberá-la adequadamente após ela não ser mais necessária (MITRE Corporation, 2026b). Isso torna o bloco de memória indisponível para realocação ou reúso.

De acordo com MITRE Corporation (2026b), é frequentemente causada por caminhos de execução, como condições de erro, dados malformados ou sessões interrompidas abruptamente, onde o código falha em executar a rotina de limpeza. Também surge da confusão sobre qual parte do programa é responsável por realizar o *free* em linguagens de gerenciamento manual, como C.

Embora normalmente resulte em problemas gerais de confiabilidade do *software*, se um invasor conseguir desencadear repetidamente esse vazamento de forma intencional, ele poderá esgotar a memória do sistema (MITRE Corporation, 2026b). Consequentemente leva a uma Negação de Serviço, em inglês *Denial of Service* (DoS) por consumo excessivo de resursos (CPU ou memória) ou travamentos.

A melhor forma é utilizar linguagens, bibliotecas ou *frameworks* que forneçam gerenciamento automático de memória (como *garbage collectors*) (MITRE Corporation, 2026b). Linguagens como C, contudo, não oferecem esse gerenciamento de forma nativa. A adoção de bibliotecas externas para suprir essa falta exige cautela do projetista, que deve ponderar o custo de desempenho (*overhead*), o risco de introdução de novas vulnerabilidades pela própria biblioteca e o aumento no esforço de análise de segurança do sistema.

2.2.2.4 CWE-415

A CWE-415, chamada de Liberação Dupla, do inglês *Double Free*, ocorre quando o produto chama a função de liberação (como `free()`) duas vezes no mesmo endereço de memória (MITRE Corporation, 2026c).

Semelhante ao vazamento de memória (2.2.2.3), costuma ser resultado de condições de erro não tratadas corretamente ou confusão lógica sobre a responsabilidade de limpar a memória, especialmente quando ponteiros são usados de forma global ou espalhados por centenas de linhas e múltiplos arquivos (MITRE Corporation, 2026c).

Chamar a função `free()` de forma dupla corrompe as estruturas de dados de gerenciamento de memória do sistema. O programa pode travar, ou, pior, fazer com que duas chamadas futuras da função `malloc()` retornem exatamente o mesmo ponteiro (MITRE Corporation, 2026c). Se o invasor controlar os dados inseridos nessa memória "duplamente alocada", ele ganha uma condição conhecida como *write-what-where* e pode realizar ataques de *buffer overflow*, chegando a **executar código arbitrário** (MITRE Corporation, 2026c).

Para garantir que cada alocação seja liberada apenas uma vez, uma técnica prática é definir o ponteiro como `NULL` imediatamente após o primeiro `free`, garantindo que não possa ser liberado novamente por engano (MITRE Corporation, 2026c).

2.2.2.5 CWE-416

A CWE-416, chamada de Uso Após Liberação, do inglês *Use After Free* (UAF), ocorre quando o programa continua a utilizar ou referenciar um ponteiro para a memória que já foi liberada (MITRE Corporation, 2026d). Como a memória já foi devolvida, ela pode ter sido realocada e preenchida com dados novos; logo, as operações realizadas com o ponteiro original interagem acidentalmente com os dados da nova alocação (MITRE Corporation, 2026d).

Frequentemente resulta de outras fraquezas primárias, como, em concorrência, condições de corrida, manipulação inadequada de erros ou apenas liberação precoce por falha lógica (MITRE Corporation, 2026d).

É possível causar alteração não intencional de memória, corrompendo dados válidos, vazamento de informações confidenciais, como permissão de leitura indevida, e DoS por travamento (MITRE Corporation, 2026d). Além disso, se o invasor conseguir inserir dados maliciosos logo após o ponteiro ser liberado, é possível **executar códigos ou comandos arbitrários** assumindo o controle da execução (MITRE Corporation, 2026d).

Além do uso de gerenciamento automático, usar o conceito visto em CWE-415 (2.2.2.4), definindo os ponteiros como `NULL` imediatamente após serem liberados. Se houver uma falha que tente reutilizar esse ponteiro depois disso, ocorrerá um travamento (Desreferenciamento de Nulo), mas isso elimina a possibilidade crítica de execução de código malicioso (MITRE Corporation, 2026c).

2.2.2.6 CWE-457

A CWE-457, chamada de Uso de Variável Não Inicializada, do inglês *Use of Uninitialized Variable*, ocorre quando o código usa uma variável que não recebeu um valor inicial (MITRE Corporation, 2026e). Em linguagens como C, variáveis armazenadas na pilha (*stack*) não são limpas por padrão e conterão lixo de memória deixado por chamadas de funções anteriores.

Segundo MITRE Corporation (2026e), tipicamente ocorre por erros tipográficos no código ou porque um fluxo lógico (como `switch` ou `if/else`) falhou em cobrir todos os casos e não atribuiu valor à variável antes de ela ser utilizada.

Essa fraqueza pode gerar comportamentos totalmente imprevisíveis, corromper o fluxo de controle esperado do programa ou causar DoS. Em cenários críticos, um invasor pode influenciar os dados residuais na pilha (pré-inicializando o lixo intencionalmente) para contornar checagens de senha, burlar travas lógicas ou **executar código** (MITRE Corporation, 2026e). O problema se agrava com *strings*, pois funções contam com um terminador nulo que pode não existir no lixo residual.

Sempre garantir explicitamente que variáveis essenciais recebam um valor padrão aceitável no momento de declaração (MITRE Corporation, 2026e). É fortemente recomendado configurar e usar ferramentas de compilação em modos rigorosos, pelos compiladores moderrnos costumarem emitir alertas consistentes quando variáveis são lidas sem serem inicializadas (MITRE Corporation, 2026e).

2.2.2.7 CWE-476

A CWE-476, chamada de Desreferenciamento de Ponteiro Nulo, do inglês *Null Pointer Dereference*, ocorre quando o código tenta desreferenciar, acessar o conteúdo ou método

apontado, um ponteiro sob a suposição de que ele é válido, mas seu valor real é `NULL` (MITRE Corporation, 2026f).

Muitas funções de busca ou resolução de rede (por exemplo: `gethostbyaddr`) retornam nulo quando falham, e os desenvolvedores se esquecem de checar o retorno (*Unchecked Return Value*) antes de usar o resultado do ponteiro (MITRE Corporation, 2026f). Também pode ser consequência de variáveis não inicializadas ou concorrência em sistemas assíncronos.

Segundo MITRE Corporation (2026f), a consequência geralmente é o **travamento completo do processo** (DoS), sendo muito difícil que o programa se recupere para um estado seguro de funcionamento, mesmo com tratamentos de exceção. Em raras circunstâncias, como quando `NULL` equivale ao endereço de memória literal `0x00` e o código privilegiado pode acessá-lo, um invasor pode ler/escrever na memória e **executar códigos** (MITRE Corporation, 2026f).

Validar e checar estritamente resultados de todas as funções, confirmando que o valor não é nulo antes de utilizá-lo. Inicializar variáveis com valores padrões também previnem essa situação (MITRE Corporation, 2026f).

~~*Common* Vulnerabilities and Exposures~~

(Seção movida para antes das CWEs, Seção 2.2.2.1.)

~~A Enumeração Comum de Vulnerabilidades e Exposições, mais popularmente conhecida do inglês *Common* Vulnerabilities and Exposures (CVE). Similarmente à CWEs, CVE atua como um dicionário, com foco em identificar, definir e catalogar vulnerabilidades de segurança cibernética que foram divulgadas publicamente (CVE Program, 2026).~~

~~CVE possui um único registro no seu catálogo para cada vulnerabilidade descoberta para uma versão específica de um *software*. O objetivo principal de se atribuir um CVE é permitir que profissionais de TI, ferramentas automatizadas e equipes de segurança cibernética saibam que estão se referindo exatamente ao mesmo problema (CVE Program, 2026).~~

~~Após a confirmação de uma vulnerabilidade, ela recebe um identificador único composto pelo ano de relato ou atribuição, seguido por um sequencial numérico. Parceiros como *National* Vulnerability Database (NVD) adicionam o registro com suas descrições, atribuindo um índice de severidade metrificado pela *Common* Vulnerability Scoring System (CVSS) e sua causa raiz proveniente da CWE (CVE Program, 2026).~~

2.2.3 Linguagem Natural

Um sistema formal amplamente utilizado para modelar a estrutura constituinte em linguagens naturais é a Gramática Livre de Contexto (*Context-Free Grammar* - CFG).

Também conhecidas como gramáticas de estrutura de frase (*phrase-structure grammars*), as CFGs utilizam um formalismo que é equivalente à Forma de Backus-Naur (BNF), padrão universal para a especificação da sintaxe em computação (JURAFSKY; MARTIN, 2026).

Uma CFG consiste em um conjunto de regras ou produções, onde cada uma expressa como os símbolos podem ser ordenados e agrupados, e um léxico de palavras e símbolos. Tal estrutura define uma linguagem formal, na qual existe uma separação estrita: sentenças que podem ser derivadas pelas regras são consideradas gramaticais, enquanto as que não podem são denominadas agramaticais (JURAFSKY; MARTIN, 2026).

Contudo, essa distinção rígida é uma característica das gramáticas formais e serve apenas como um modelo simplificado das linguagens naturais. Diferente das linguagens de programação, a validade e o sentido em linguagem natural dependem intrinsecamente do contexto. Na linguística computacional, o uso desses modelos para capturar tal complexidade é fundamentado na gramática generativa, visto que a linguagem passa a ser definida pelo conjunto de possíveis 'geradas' por suas regras gramaticais (JURAFSKY; MARTIN, 2026).

2.2.3.1 Processamento de Linguagem Natural

O Processamento de Linguagem Natural, do inglês *Natural Language Processing* (NLP), tem de objetivo fundamental capacitar os sistemas computacionais a processar, armazenar, compreender e gerar a linguagem humana, permitindo que as máquinas se comuniquem com sucesso e realizem tarefas inferenciais complexas (JURAFSKY; MARTIN, 2026).

Os autores pontuam que o problema da compreensão da linguagem é consideravelmente complexo, pois não se restringe apenas ao entendimento da estrutura sintática das sentenças, mas exige uma profunda compreensão da semântica, do contexto e do assunto abordado.

O campo de NLP progrediu através de diferentes abordagens de resolução, a abordagem mais relevante para este trabalho é o Aprendizado Profundo (*Deep Learning*). Baseado na arquitetura *Transformer*, esse paradigma mudou o foco da engenharia manual de características para a aprendizagem de representação (*representation learning*), onde os modelos aprendem automaticamente vetores de significado contínuos (*embeddings* contextuais) operando em vastas quantidades de texto não rotulado, mediante o pré-treinamento autossupervisionado (JURAFSKY; MARTIN, 2026).

O NLP é dividido em tarefas de Compreensão de Linguagem Natural (*Natural Language Understanding* - NLU), focadas em extrair significado, e Geração de Linguagem Natural (*Natural Language Generation* - NLG), focadas na produção de texto coerente (TUNSTALL; WERRA; WOLF, 2022).

Para modelos mais utilizados de LLMs, o foco inerente desses modelos é NLG. Graças à capacidade de aprendizado em contexto (*in-context learning*) guiado por *prompts*, esses modelos utilizam a sua capacidade de geração (NLG) para resolver, de forma indireta, os

problemas de compreensão (NLU) (TUNSTALL; WERRA; WOLF, 2022; JURAFSKY; MARTIN, 2026).

2.2.4 Arquitetura *Transformer*

A arquitetura *Transformer*, proposta por Vaswani et al. (2017) inicialmente para tarefas como a tradução automática, é um modelo de transdução de sequências que dispensa o uso total de Redes Neurais Recorrentes (*Recurrent Neural Networks* RNNs) e convolucionais (*Convolucional Neural Networks* - CNNs). Ao invés disso, ela se baseia exclusivamente em mecanismos de atenção para capturar dependências globais entre as sequências de entrada e saída.

A arquitetura segue uma estrutura de **Codificador-Decodificador** (*Encoder-Decoder*).

- **Codificador (*Encoder*)**: Segundo Vaswani et al. (2017), o codificador tem a função de converter uma sequência de *tokens* de entrada em uma sequência de representações contínuas (*embeddings* ou estados ocultos contextuais). Composto por uma pilha de camadas idênticas de $N = 6$. Cada camada possui duas subcamadas principais:

  1. Mecanismo de Autoatenção de Múltiplas Cabeças (*Multi-Head Self-Attention*).
  2. Rede Neural *Feed-Forward* totalmente Conectada, aplicada de forma independente a cada posição (*Position-Wise fully Connected Feed-Forward Network*).

  Em torno de cada uma das subcamadas, o modelo emprega uma conexão residual (*skip/residual connection*) seguida de uma normalização de camada (*layer normalization*) (VASWANI et al., 2017; TUNSTALL; WERRA; WOLF, 2022).

  Esses modelos convertem uma sequência de texto de entrada em uma representação numérica, sendo muito eficientes para tarefas como classificação de texto ou reconhecimento de entidades nomeadas. A representação calculada para cada *token* nessa arquitetura depende tanto do contexto à esquerda (antes do *token*) quanto à direita (depois do *token*); esse mecanismo é geralmente denominado atenção bidirecional (TUNSTALL; WERRA; WOLF, 2022).

- **Decodificador (*Decoder*)**: O decodificador utiliza as representações contínuas geradas pelo codificador para produzir iterativamente a sequência de saída, gerando um símbolo (*token*) de cada vez, com uma propriedade autorregressiva, ou seja, consumindo os símbolos gerados anteriormente como entrada adicional (VASWANI et al., 2017; TUNSTALL; WERRA; WOLF, 2022).

  Assim como o codificador, é composto por uma pilha de $N = 6$ camadas idênticas. Além das camadas presentes no codificador, o decodificador introduz uma terceira subcamada e aplica uma modificação vital na sua autoatenção:

1. Autoatenção Mascarada de Múltiplas Cabeças (*Masked Multi-Head Attention*): A autoatenção do decodificador é mascarada (através da atribuição de valores $-\infty$ para conexões ilegais) para impedir que uma posição acesse informações de *tokens* futuros (*leftward information flow*), garantindo que a previsão do próximo *token* dependa apenas das saídas já conhecidas (VASWANI et al., 2017; TUNSTALL; WERRA; WOLF, 2022).
2. Atenção Codificador-Decodificador (*Encoder-Decoder Attention* ou *Cross-Attention*): Esta subcamada realiza a atenção sobre a saída da pilha do codificador. Nela, as consultas vêm da camada anterior do decodificador, enquanto as chaves e valores vêm da saída do codificador, permitindo que o modelo relacione *tokens* de ambas as sequências (VASWANI et al., 2017; TUNSTALL; WERRA; WOLF, 2022; JURAFSKY; MARTIN, 2026).
3. Rede Neural *Feed-Forward* orientada à posição.

A partir disso, dada uma entrada de texto, esses modelos completam a sequência prevendo iterativamente a palavra subsequente mais provável.

A função de atenção no *Transformer* mapeia um vetor de Consulta (*Query* - Q), um conjunto de vetores Chave (*Key* - K) e vetores Valor (*Value* - V) para produzir uma saída (VASWANI et al., 2017; TUNSTALL; WERRA; WOLF, 2022).

A Atenção de Produto Escalar Escalado (*Scaled Dot-Product Attention*) é calculada pela seguinte fórmula:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

onde d_k é a dimensão dos vetores Chave. Essa operação permite que cada *token* "preste atenção" em todos os outros *tokens* da sequência, ponderando sua importância relativa. O fator de escala $\sqrt{d_k}$ é utilizado para estabilizar os gradientes durante treinamento.

Esses modelos não aplicam a atenção apenas uma vez. Em vez de realizar uma única função de atenção com dimensões globais, a arquitetura projeta as Consultas, Chaves e Valores h vezes de forma linear e independente. Eles realizam h projeções lineares independentes de Q, K e V. Espressada por:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O \tag{2.2}$$

onde $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$. Isso permite que o modelo atenda simultaneamente a informações de diferentes subespaços de representação em várias posições do texto (VASWANI et al., 2017).

Dado que o *Transformer* não contém recorrência ou convolução, o modelo nativamente desconhece a ordem de sequência. Para solucionar isso, Vaswani et al. (2017) propuseram injetar informações sobre a posição relativa ou absoluta dos *tokens* na sequência através

de Codificações Posicionais (*Positional Encodings*) (VASWANI et al., 2017). Esses vetores de posição possuem a mesma dimensão que os *embeddings* dos *tokens* e são inseridos nas bases das pilhas do codificador e do decodificador.

No final da pilha do decodificador, o vetor de representação contínua é passado por uma transformação linear (*unembedding matrix*) e, em seguida, por uma função *softmax* para converter as pontuações (*logits*) em probabilidades normalizadas sobre o vocabulário, gerando a previsão do próximo *token* (VASWANI et al., 2017).

2.2.4.1 Modelos de Linguagens de Grande Escala

Os Modelos de Linguagem de Grande Escala, do inglês *Large Language Models* (LLMs), são sistemas computacionais baseados em redes neurais profundas, quase exclusivamente utilizando a arquitetura *Transformer*, projetados para prever, gerar e interagir utilizando Linguagem Natural (2.2.3) (JURAFSKY; MARTIN, 2026).

Segundo Jurafsky e Martin (2026), uma LLM é um sistema probabilístico que, dado um contexto ou prefixo de palavras, atribui uma distribuição de probabilidade aos possíveis próximos símbolos (*tokens*) do vocabulário.

Existem três configurações principais de *Transformers* (codificadores, decodificadores e codificadores-decodificadores), os LLMs generativos modernos (como a família GPT, Claude, Lhama, etc.) utilizam a arquitetura baseada estritamente em Decodificadores (*Decoders*). Pela natureza de gerar iterativamente o próximo *token* por vez e consumir a própria saída como entrada para o passo seguinte, esse processo é denominado geração causal ou autorregressiva (*causal or autorregressive generation*) (JURAFSKY; MARTIN, 2026).

A ideia central, proposta por Jurafsky e Martin (2026), que torna os LLMs possíveis é o pré-treinamento. Os modelos aprendem conhecimentos profundos sobre a linguagem, o contexto e o mundo simplesmente sendo ensinados de forma iterativa a prever a próxima palavra em *corpora* gigantescos de texto.

Esse processo é autossupervisionado (*self-supervised*), pois a sequência natural de palavras serve como sua própria supervisão. O algoritmo ajusta bilhões de parâmetros da rede (pesos de atenção e *feed-forward*) minimizando a função de perda de entropia cruzada (*cross-entropy loss*), que penaliza o modelo quando ele atribui uma probabilidade baixa à palavra que de fato é a correta no texto (JURAFSKY; MARTIN, 2026).

Após o pré-treinamento, qualquer tarefa de Processamento de Linguagem Natural (2.2.3.1) pode ser modelada como uma geração condicional guiada por um *prompt* (JURAFSKY; MARTIN, 2026). Sua capacidade de aprendizado em contexto (*in-context learning*) permite que LLMs aprendam a executar novas tarefas durante a fase de inferência (quando estão apenas gerando texto) a partir das instruções ou demonstrações contidas no *prompt*, sem que haja nenhuma atualização de parâmetros baseada em gradientes no modelo subjacente (JURAFSKY; MARTIN, 2026).

Um modelo apenas pré-treinamento tende a simplesmente dar continuidade ao texto de forma passiva em vez de responder à intenção do usuário, além de ser suscetível a gerar alucinações (*hallucinations*) ou saída inseguras (JURAFSKY; MARTIN, 2026). Para torná-los assistentes úteis, os LLMs passam por duas fases cruciais de Pós-Treinamento (*Post-training*).

Primeiro, o modelo é submetido a um ajuste fino (*finetuning*) utilizando um conjunto de dados contendo pares de instruções e as respostas desejadas. Ensinando o modelo não apenas a resolver tarefas, mas atua como um meta-aprendizado para que ele se torne fundamentalmente melhor em seguir instruções de modo geral (JURAFSKY; MARTIN, 2026).

Para garantir que o modelo seja útil e inofensivo (*helpful and non-harmful*), técnicas de Aprendizado por Reforço (*Rein-forcement Learning* - RL) ou Otimização Direta de Preferências (*Direct Preference Optimization*) são utilizadas. O modelo é treinado para que suas saídas maximizem uma recompensa alinhada com os valores e preferências humanas, penalizando respostas indesejadas, rudes ou incorretas (JURAFSKY; MARTIN, 2026).

2.2.4.2 Agentes e Sistemas Multi-Agentes

Um agente é definido como qualquer entidade capaz de perceber o seu ambiente através de sensores e agir sobre esse ambiente por meio de atuadores (RUSSELL; NORVIG, 2020). O comportamento lógico dessa entidade é matematicamente descrito por uma função de agente, que mapeia a sequência completa de percepções recebidas para a escolha de uma ação correspondente (RUSSELL; NORVIG, 2020).

O objetivo no desenvolvimento dessas soluções é a construção de um agente racional, ou seja, aquele que seleciona ações com a expectativa de maximizar sua medida de desempenho, baseando-se nas evidências do ambiente e em seu conhecimento embutido (RUSSELL; NORVIG, 2020).

A complexidade de determinados problemas, contudo, exige soluções que transcendem a capacidade de um único agente. Nestes casos, adota-se a arquitetura de Sistemas Multi-Agentes (RUSSELL; NORVIG, 2020). Os desafios de *design* em ambiente multi-agentes diferem significativamente dos ambientes isolados, uma vez que, em cenários cooperativos, a comunicação e a coordenação entre as entidades emergem como comportamentos racionais necessários para evitar redundâncias e atingir o objetivo de forma eficiente (RUSSELL; NORVIG, 2020).

Com a consolidação dos Modelos de Linguagem de Larga Escala (LLMs), o paradigma baseado em agentes ganhou nova capacidade operacional. As LLMs passaram a atuar como o núcleo de raciocínio dos agentes, permitindo que interajam com o mundo, gerem raciocínios complexos e invoquem ferramentas externas de forma autônoma.

Em um *framework* multi-agente fundamentado em LLMs, o conhecimento do modelo de base é particionado: cada agente atua com instruções e metas específicas (o *system*

prompt) (RUSSELL; NORVIG, 2020). A operação colaborativa de múltiplos agentes LLM atenua o risco de alucinações e o gargalo das janelas de contexto, pois o problema global é fracionado em sub-tarefas iterativas de resolução mais simples e verificável.

2.2.5 Métricas de Avaliação

As métricas de avaliação para uma ferramenta de análise estática funcional requer métricas que permitam analisar não apenas a acurácia geral, mas o equilíbrio entre erros de falso positivo e falso negativo. Neste trabalho, serão utilizadas diferentes métodos de avaliação, se dividindo em Validade Gramatical (2.2.5.2), Validade Funcional (2.2.5.3), Validação em *Dataset* (2.2.5.4), Validação no mundo real (2.2.5.5), Custo (2.2.5.6).

2.2.5.1 Juliet Test Suite

O *Juliet Test Suite* é uma coleção de programas escritos em C/C++ e Java, contendo casos de teste sintéticos, que foram desenvolvidas como exemplos de vulnerabilidades (NSA Center for Assured Software, 2017).

A coleção foi desenvolvida por NSA Center for Assured Software (2017), um projeto de domínio público. A versão mais recente 1.3 para C/C++ possui mais de 64 mil casos de teste espalhados em mais de 100 mil arquivos. O *dataset* não foca em problemas lógicos ou de arquitetura geral de *software*, mas em vulnerabilidades de código categorizadas pelo sistema CWE (2.2.2.2).

O propósito central é servir como um "campo de provas" padronizado para ferramentas de análise de código, como analisadores estáticos (do inglês *Static Application Security Testing* - SAST). Esse conjunto será utilizado para analisar todos os CWEs selecionados para este trabalho, analisando a eficácia das regras produzidas.

2.2.5.2 Validade Gramatical

Determina se uma regra é identificada com sucesso pela ferramenta de análise estática sem gerar erros de sintaxe.

Para avaliar a validade de sua gramática, cada regra é aplicada na ferramenta para avaliar um código teste, avaliando somente se a ferramenta reporta ou não algum erro de sintaxe na própria regra durante a execução.

Sendo analisado também quais regras geram *loops* de correção, quais o agente de correção não foi capaz de corrigir.

2.2.5.3 Validade Funcional

Define a precisão da regra sobre distinguir corretamente um código defeituoso e corrigido. Para realizar essa avaliação, cada regra introduzida à ferramenta, é avaliado os relatórios gerados a partir dela sobre o código gerador da regra (defeituoso e corrigido).

Analisando a verificação em ambos casos, é identificado o número de relatos no código com *bugs* ($X_{identificados}$) e no código corrigido ($X_{corrigidos}$), sendo validado somente nos casos em que no código corrigido não seja identificado nenhum relatório ($X_{corrigidos} = 0$) e no código defeituoso todos os N casos presentes ($X_{identificados} >= N$).

2.2.5.4 Validação em *Dataset*

A métrica mais intuitiva para avaliar um modelo é a Acurácia (*Accuracy*), medindo a proporção de previsões corretas em relação ao total de previsões. Porém, como em casos de aplicação de mundo real muitos códigos não estão categorizados e corretamente identificados, é perigosamente enganosa a sua métrica neste escopo.

A alternativa é o **F1-Score**, sendo uma métrica utilizada para avaliar o desempenho de modelos de classificação. Cada previsão feita pelo modelo cai em uma das quatro categorias:

- **Verdadeiros Positivos** (*True Positive* - TP);
- **Falsos Positivos** (*False Positive* - FP);
- **Falsos Negativos** (*False Negative* - FN);
- **Verdadeiros Negativos** (*True Negative* - TN).

A **Precisão** (*Precision - P)* indica a proporção de exemplos positivos que foram corretamente identificados. No contexto desse trabalho, representa a proporção de trechos de códigos classificados que realmente eram *bugs*. É especialmente importante sua corretude quando o custo de falsos positivos é alto, causando a inutilização da ferramenta (AHO et al., 2007).

$$Precision = \frac{TP}{TP + FP} \tag{2.3}$$

A **Revocação ou Sensibilidade** (*Recall* - R) mede a capacidade do modelo de encontrar todas as instâncias positivas reais. Neste caso, indica quantas das vulnerabilidades foram realmente identificadas pelo modelo.

$$Recall = \frac{TP}{TP + FN} \tag{2.4}$$

Avaliou-se especificamente se a ferramenta foi capaz de apontar o alerta diretamente nos pontos do programa já rotulados e conhecidos como vulneráveis.

A métrica **F1-Score** representa a média harmônica entre a Precisão e Revocação, sendo particularmente útil quando se busca um equilíbrio entre as duas. Sua fórmula é calculada por:

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{2.5}$$

No cenário da Análise Estática, o F1-Score reflete competentemente o *trade-off* entre as identificações corretas e falhas.

2.2.5.5 Validação no mundo real

Na aplicação para projetos de código aberto, é desconhecido o número de vulnerabilidades. A partir desse conhecimento, este trabalho adotou as mesmas métricas utilizadas por Li et al. (2026), a **Taxa de Falsas Descobertas Amostradas** (do inglês *Sampled False Discovery Rate* - SFDR).

Na execução da ferramenta em projetos *open-source* sem rótulos conhecidos, é extraído o **Número de Relatórios** ($\#Relatorios$) gerados, o **Número de Arquivos** ($\#Arquivos$) afetados, contendo pelo menos um aviso e **Número de Falsos Positivos** ($\#FPs$), instâncias incorretamente relatadas como vulnerabilidades. Baseado nessas categorias, estimamos o SFDR como:

$$SFDR = \frac{\#FPs}{\#Relatorios} \tag{2.6}$$

A partir desses dados, é possível avaliar o quão eficiente será a ferramenta em problemas no mundo real.

2.2.5.6 Custo

Para analisar de forma adequada o custo gerado pela geração e análise, este trabalho utiliza as mesmas avaliações adotadas na área. Será analisado o número de consumo de *tokens* processados nas entradas e saídas para cada projeto, bem como a medição do tempo exato de execução que a ferramenta levou para analisar o repositório.

2.3 Trabalhos correlatos

~~Os três trabalhos apresentados a seguir foram selecionados por enfrentarem diretamente o mesmo problema central: a análise estática tradicional, buscando melhorias através de LLMs e geração de regras utilizando a mesma ferramenta SemGrep. Cada um deles avança com seus próprios *pipelines* na utilização da LLM, cada um com um papel diferente no processo de geração de regras.~~

Os três trabalhos apresentados a seguir foram selecionados por enfrentarem diretamente o mesmo problema central: as limitações da análise estática tradicional na detecção de vulnerabilidades, buscando superá-las com o apoio de LLMs. Cada um deles atribui à

LLM um papel diferente: sintetizar verificadores para um analisador existente (YANG et al., 2025), gerar regras para o SemGrep (HOU et al., 2025) ou auditar diretamente o repositório por meio de um agente autônomo (GUO et al., 2025).

2.3.1 Trabalho 1: *KNighter: Transforming Static Analysis with LLM-Synthesized Checkers* (YANG et al., 2025)

O trabalho de Yang et al. (2025) é considerado o estudo pioneiro para a área de Analisadores Estáticos Automatizados. Publicado em SOSP 25 *(ACM SIGOPS 31st Symposium on Operating Systems Principles)*, em Seul, República da Coreia. KNighter foi o primeiro sistema de geração de analisadores estáticos totalmente automatizado (YANG et al., 2025).

Os autores buscam resolver o problema principal com métodos de análise tradicional. Apesar da análise estática ser o método mais eficiente pela sua falta de execução de código, para gerenciamento de sistemas de larga escala, esses métodos falham na falta de identificar diferentes padrões de bugs e gerenciamento de bases enormes de código (YANG et al., 2025). A figura 1 mostra a motivação que os autores tinham sobre analisadores automatizados.

Motivado pelas limitações expostas, o trabalho propõe um *framework* voltado à extração de padrões a partir de histórico de *commits*. Como exemplo empírico, a identificação de chamadas à função `devm_kzalloc` desacompanhadas de verificação de nulidade configura uma heurística válida para a detecção da vulnerabilidade *Null Pointer Dereference* (CWE-476) (YANG et al., 2025).

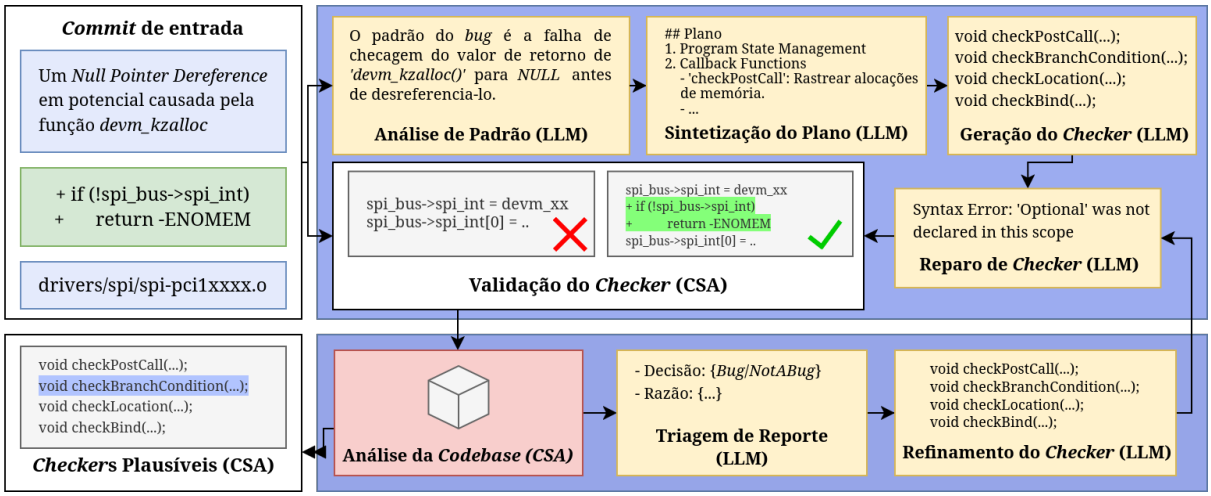
Com um *commit* como entrada, KNighter retorna como saída um analisador gerado para *Clang Static Analyser* (CSA). Sendo um verificador válido aquele que identifica o código pré alteração do *commit* como *bug* e o código corrigido como correto (YANG et al., 2025).

O motor principal do framework é *Clang Static Analyser* (CSA), Yang et al. (2025) explica que CSA opera a partir de uma execução simbólica *path-sensitive*, construindo uma representação interna chamada *ExplodedGraph*. CSA é modulável pela sua construção a partir de *checkers*, sendo eles, pequenos componentes especializados, montados a partir de um *template* (YANG et al., 2025).

Na figura 7 mostra como o KNighter funciona em duas fases: sintetização de *checkers*, o projeto analisa o *patch* de entrada para identificar padrões de *bugs*, sintetiza, a partir do conteúdo identificado, um plano de detecção e implementa um CSA *checker* (YANG et al., 2025). Se ocorre qualquer erro de compilação, é encaminhado para um agente de sintaxe que automaticamente repara o *checker* a partir das mensagens de erro (YANG et al., 2025).

No que tange à implementação do trabalho, a metodologia descrita por Yang et al.

Figura 7 – *Overview* do KNighter



Fonte: Adaptado para o português para melhor compreensão (YANG et al., 2025).

(2025) fundamenta-se na coleta de *commits*, viabilizando uma análise comparativa e estrutural entre versões vulneráveis de código-fonte e suas respectivas correções. Foram coletados 10 tipos de CWEs, criando para cada CWE um respectivo CSA *checker*.

Além dessa coleção, foram preparados 3 tipos de exemplos de *checkers* para aprendizado *in-context*, lidando com os respectivos CWEs: *Null Pointer Dereference* (CWE-476), *Use of Uninitialized Variable* (CWE-457) e *Double Free* (CWE-415) (YANG et al., 2025). Além desses exemplos, os autores identificaram repetições de atividades que cada *checker* reutiliza, então separaram em 9 *sub-checkers* para poderem serem reutilizados em outros verificadores.

Para a avaliação de cada *checker*, é analisado sua eficácia para o *bug* do *commit* analisado e reconhecer sua correção. No repositório original do *patch*, o analisador realiza uma varredura integral na versão do código imediatamente à correção, quantificando o total de vulnerabilidades identificadas (N_{buggy}).

Em seguida, uma nova varredura é executada na base atualizada para contabilizar as vulnerabilidades remanescentes (N_{patched}), viabilizando assim a validação da eficácia do verificador. Um *checker* é considerado válido se $N_{\text{buggy}} > N_{\text{patched}}$ e $N_{\text{patched}} < T_{\text{valid}}$, onde T_{valid} é um valor limite (padrão 50) (YANG et al., 2025).

Apesar de seu forte pioneirismo, Yang et al. (2025) analisa suas falhas com seu projeto, com 36,06% dos *commits* não sendo possível gerar um analisador a partir disso, com sua maioria falhando por implementações incorretas, com o problema comum são otimizações realizadas pelo respectivo compilador (como strcp e memset), impedindo que o *checker* lidasse com essas chamadas.

Além disso, notaram limitações relacionadas a *Buffer Overflow* e *Use of Uninitialized Variable*. Os autores atribuem essa dificuldade a dois fatores principais. O primeiro é a limitação inerente da análise estática em determinar valores precisos, principalmente

quando é preciso determinar limites de *buffers*. O segundo é a dificuldade para analisar códigos *multi-thread*, como no gerenciamento correto de *locks* (YANG et al., 2025).

KNighter relata dentre 37 *checkers* plausíveis, foram relatados 92 novos *bugs*, que depois de uma inspeção manual, 32,2% foram falsos positivos (YANG et al., 2025). Sendo o principal erro que envolve os analisadorescom seu padrão correto e regras para *matching* corretos, porém o *checker* falha em gerenciar *triggers* ou a condição do estado corretamente (como identificar um ponteiro já verificado antes de seu uso) (YANG et al., 2025).

Para o presente trabalho, a contribuição de Yang et al. (2025) é insubstituível no ponto de vista conceitual: a automatização para um sistema de análise estática, gerenciamento de mais de um agente para refinamento e correção, cobrindo CWEs relevantes adotados neste TCC.

No entanto, a ferramenta auxiliar difere: enquanto Yang et al. (2025) opera gerenciando regras para CSA, este trabalho opera exclusivamente com SemGrep, com suas regras de inclusão e exclusão lógica auxilia para evitar falsos positivos, que foi um defeito crítico em KNighter. Além disso, KNighter utiliza somente *commits* para seu treinamento, o que limita sua detecção pelo simples fato de existirem outras vulnerabilidades que não se encaixam nos *patches* já corrigidos, então para contribuir na solução deste problema, a utilização de *datasets* como NSA Center for Assured Software (2017) e *dataset* personalizado de Li et al. (2026).

2.3.2 Trabalho 2: *Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models* (HOU et al., 2025)

O trabalho de Hou et al. (2025), publicado no IEEE International Conference on Software e Quality, Reliability and Security (QRS) em 2025, representa um dos trabalhos que compõem o atual estado da arte em Análise Estática Automatizado até o momento da elaboração desse TCC. Com a motivação de melhorar a análise estática para *bugs* contextualizados de específicos projetos, utilizando *rulesets* lidando com o máximo de casos (*corner-cases*) diminuindo o trabalho braçal e capacidade de migrar regras para diferentes linguagens (HOU et al., 2025).

Os autores citam KNighter, explicando que apesar de sua efetividade, KNighter não possui dinâmica para lidar com diferentes linguagens, exigindo diversas alterações manuais para conseguir seu funcionamento (HOU et al., 2025). Propondo um novo método baseado em LLMs que gera regras de análise estática a partir de padrões de históricos de código, utilizando regras gramáticas e *few-shot learning* em contexto, além de suporte para migração de regras existentes para outras linguagens, geração de exemplos para cada regra especificado para lidar com *corner-cases* (HOU et al., 2025).

Arquiteturamente, o *framework* proposto adota uma estratégia de *few-shot learning* para

administrar conhecimento do domínio adquirido das LLMs durante seu pré-treinamento não supervisionado, integrando especificações de sintaxe de regras, histórico de informações de erros e descrição de otimizações de regras para guiar o comportamento de geração de regras das LLMs (HOU et al., 2025). Hou et al. (2025) apontam que isso efetivamente soluciona dificuldades em *corner-cases* que seriam mais complexos para um humano criar, diminuindo o caso de falsos positivos e falsos negativos.

A ferramenta de análise estática utilizada, para reduzir o uso de apenas LLMs, é SemGrep. Os autores explicam que análise estática baseada em regras proveem extensibilidade por suas regras customizaveis, permitindo checagens para problemas específicos para projetos específicos (HOU et al., 2025).

Consequentemente, Hou et al. (2025) afirma que não existe necessidade da construção de um analisador específico para realizar essas checagens. Para o objetivo do artigo com migração de regras, SemGrep se encaixa perfeitamente por seu suporte a mais de 30 linguagens, mimetizando qualquer das linguagens suportadas com seu arquivo YAML (HOU et al., 2025).

Do ponto de vista técnico, a abordagem proposta segue um *workflow* de três estágios: a Geração, Migração e Otimização das regras. No estágio inicial de Geração, o método dos autores é reconhecimento de *patches* de código e gera descrições das regras, assim gera a regra a partir da descrição. Com as regras geradas, cada uma é avaliada em *snippets* de teste para verificar uma eficácia, então as descrições das regras podem ser otimizadas de acordo com falsos positivos e falsos negativos (HOU et al., 2025).

Para o segundo estágio, seu método gera descrições de linguagens naturais e seus *snippets* de teste para a migração da regra, subsequentemente criando a regra. No último estágio, seu método lê a regra criada, independente se foi gerada ou não, e cria *snippets* para lidar com *corner-cases*, seguindo os mesmos passos do estágio um (HOU et al., 2025).

Sua construção de metodologia segue um fluxo similar ao Yang et al. (2025), apesar de criar uma avaliação extra relacionada à migração de linguagens. Para a utilização de *datasets* para treinamento, Hou et al. (2025) utiliza, para geração de regras, foram selecionadas aleatóriamente 111 regras dentre o *ruleset* oficial do SemGrep, cada regra contendo um YAML e *snippets* de um código com *bugs* e sua correção, inserindo cada regra para seu agente realizar a descrição de cada regra, com uma validação manual para garantir que as descrições correspondiam a cada regra.

Para a adaptação de gramática do SemGrep, Hou et al. (2025) utiliza a documentação oficial do SemGrep, incluindo partes relacionadas a Sintaxe de Regras e Sintaxe de Padrões. Convertendo o texto para *Markdown*, e inserindo a gramática no contexto dos agentes. Apesar de a documentação ser feita para leitores humanos, Hou et al. (2025) inseriram códigos exemplos para um treinamento *few-shot* para sua familiarização com a gramática.

Para validar seu experimento, Hou et al. (2025) utilizam três validações.

Validação Gramática, regras com a gramática válida são aquelas que a regra gerada

é reconhecida pela ferramenta de análise estática, para essa avaliação, foi aplicado cada regra com uma amostra de código aplicado na ferramenta, e examinado se SemGrep relata algum erro de gramática para a regra (HOU et al., 2025);

Validação Funcional, uma regra é funcionalmente válida a partir que as regras geradas conseguem reportar *bugs* em uma amostra de teste. Para cada amostra de teste, contendo N *bugs* que se encaixam na respectiva regra, se a regra produzida retorna B_{VP} verdadeiros positivos e B_{FP} falsos positivos, é definido dois indicativos para avaliar a validade da regra: (HOU et al., 2025)

$$Taxa\ de\ Falsos\ Positivos : FP = \frac{B_{FP}}{N} \tag{2.7}$$

$$Taxa\ de\ Verdadeiros\ Positivos : TP = \frac{B_{VP}}{N} \tag{2.8}$$

Utilizando essa metodologia de avaliação, Hou et al. (2025) utiliza o trabalho de Yang et al. (2025) como comparação de eficácia. Como o trabalho de KNighter provê publicamente um *dataset* com 61 *commits* de *bugs fixes* no Kernel Linux, contendo o erro e sua correção, Hou et al. (2025) avalia seu método para, além de comparar com KNighter mas, avaliar sua performance no Kernel Linux.

Resultados de Hou et al. (2025) criaram 41 regras válidas comparadas com as 37 geradas por Yang et al. (2025), modestiamente ultrapassando performance contra KNighter, apesar dos autores afirmarem que seu trabalho precisa de mais refinamento. Além de comparar com KNighter, Hou et al. (2025) compara sua efetividade contra ferramentas comuns de análise estática como Smatch, CPPCheck, *ruleset* oficial do SemGrep e TScanCode. Nenhuma das ferramentas identificaram os mesmos de alguns *bugs* que sua metodologia conseguiu, que os autores dizem ser talvez pela falta de conhecimento do domínio específico do projeto analisado que métodos baseados em LLMs conseguem detectar (HOU et al., 2025).

Para o presente trabalho, o estudo de Hou et al. (2025) cumpre dois papéis fundamentais.

Primeiro, apresenta aprimoramentos comparado com o seu pioneiro KNighter com seu fluxo lógico interpretativo, se mostrando promissor utilizando ferramentas sem limites de linguagem, Hou et al. (2025) mostra que SemGrep, com suas funções lógicas, auxilia para evitar falsos positivos, apesar de seu custo computacional pelos diversos agentes.

Segundo, estabelece o referencial na utilização de ferramentas tradicionais para integração com LLMs. Mesmo sem integração de outros *datasets* para inclusão de novas regras, Hou et al. (2025) foi capaz de exceder em performance pela mudança de *workflow* para cada *commits*.

~~Trabalho 3: *LLM-based* Vulnerability Detection at Project Scale: An Empirical Study (LI et al., 2026)~~

(Trabalho substituído pelo RepoAudit (GUO et al., 2025), a seguir. A figura e as equações abaixo pertencem à versão anterior.)

O trabalho de Li et al. (2026), publicado no arXiv em 27 de janeiro de 2026, aguardando a revisão de seu trabalho para ser publicado em IEEE Transactions on Software Engineering (TSE), dentre os artigos encontrados, o trabalho mais atual na área de análise estática de vulnerabilidades automatizados via LLMs.

Sua metodologia foi pensada para preencher as seguintes lacunas na análise estática automatizada:

(1) Efetividade e Complementaridade em projetos de mundo real, métodos para fonte de regras possuem limitações em personalização. Não ficando clara a sua eficácia, tampouco o quão generalizados e robustos são, de fato, esses métodos, quanto aplicados em projetos capazes de conter diversos tipos de vulnerabilidades (LI et al., 2026);

(2) Análise Abrangente sobre Causas-Raíz de Falhas, Li et al. (2026) afirma que estudos recentes de análise estática baseada em LLMs priorizam métricas como precisão, *recall* e *F1-Score*, sem analisar sistemáticamente os fatores que afetam seu uso prático em um projeto no mundo real. Em particular, as causas-rais casos de falsos positivos são raramente categorizadas ou quantificadas, dificultando usuários em como melhorar as ferramentas (LI et al., 2026);

(3) Escalabilidade e *Overhead*, os autores explicam que uma das principais preocupações nesses métodos baseados em LLMs é seu custo e eficiência. Porém, estudos recentes raramente medem o potencial custo máximo do uso computacional desses análisadores, como contagem de *tokens* e tempo de processamento dependendo de projetos (LI et al., 2026).

A partir dessas lacunas, o trabalho de Li et al. (2026) apresenta um estudo empírico analisando 5 projetos de análise estática baseado em LLMs lidando com múltiplos tipos de CWEs, linguagens de programação e *workflows*. Sua avaliação utiliza duas configurações complementares: (i) *dataset* personalizado com 222 vulnerabilidades conhecidas em contexto de mundo real, cobrindo 8 tipos de CWEs em C/C++ e Java, utilizado para avaliar a deteecção de vulnerabilidades sistemáticamente (LI et al., 2026). (ii) 24 projetos *open-source* ativos, utilizados para avaliar em usabilidade prática no mundo real.

Em seguida, foi examinado manualmente 385 resultados de análises, com mais de 150 horas para identificar cada falso positivo e sua causa raiz, e medir o seu custo máximo ou *overhead* (LI et al., 2026).

Primeiro resultado de Li et al. (2026) foi que **métodos** existentes baseados em LLMs exibem cobertura limitada para vulnerabilidades de mundo real, utilizando seu *dataset* personalizado, os métodos avaliados atingiram um *recall* médio de **21,09%** e **33,82%** em C/C++ e Java, respectivamente, indicando que eles ainda falham em uma grande porção de vulnerabilidades, geralmente pela falta de contexto e definição contínua da variável. Apesar dessa conclusão, os autores afirmam que esses métodos, comparado com

ferramentas tradicionais, ainda descobrem mais vulnerabilidades únicas (LI et al., 2026).

Segunda conclusão foi **ambos** os métodos de análise estática (tradicionais e baseada em LLMs) exibem uma alta taxa de falsos positivos, com a melhor ferramenta analisada ainda teve **85,3%** de taxa média de falsos positivos (LI et al., 2026). Dentre a avaliação das 385 detecções, foi estudado a razão de cada falso positivo e quantificado sua distribuição dentre ferramentas, mostrando que raciocínios superficiais de fluxo de dados, a identificação imprecisa de fontes e negligência de pontos críticos do programa em contextos complexos constituem as causas-raiz predominantes para falsos positivos analisados (LI et al., 2026).

A última conclusão é que a **análise** de detecção de vulnerabilidades baseada em LLMs para projetos de larga escala continua computacionalmente cara, Li et al. (2026) realizou uma análise de custo em que mostrou que se pode custar entre cem mil *tokens* à cem milhões de *tokens* por projeto (p. ex., até 38,31 milhões de tokens de entrada e 38,07 milhões de tokens de saída), além do processamento de múltiplas horas para múltiplos dias (p. ex., até 4.638 minutos), criando uma limitação crítica para sua escalabilidade.

Com o foco na linguagem C/C++, Li et al. (2026) utilizou KNighter de Yang et al. (2025) e RepoAudit de Guo et al. (2025) e para métodos tradicionais, utilizaram SemGrep (*ruleset* padrão) e CodeQL.

Considerando as seguintes métricas de avaliação, **Com o *dataset* personalizado**, é considerado uma vulnerabilidade detectada com sucesso pela ferramenta enquanto ela reporta uma vulnerabilidade (ou seja, positivo verdadeiro: TP), caso contrário, a vulnerabilidade é despercebida pela ferramenta (ou seja, falso negativo: FN) (LI et al., 2026). Em particular, já que códigos não vulneráveis nos projetos são imensos e não totalmente identificado, Li et al. (2026) foca em *recall* do que precisão.

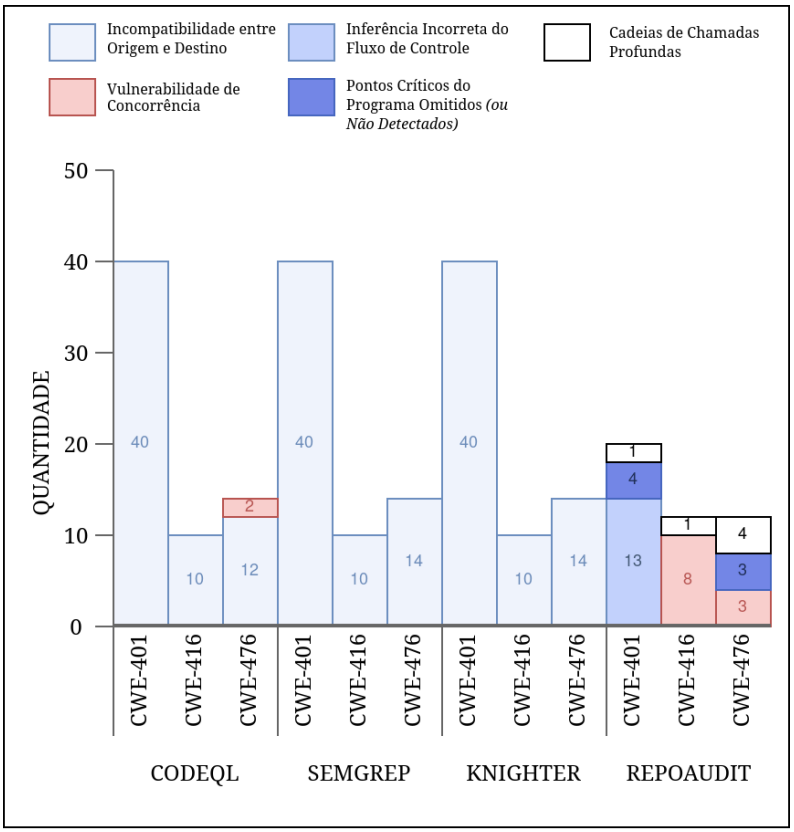
$$Recall = \frac{TP}{TP + FN} \times 100\%. \tag{2.9}$$

Dentre os projetos *open-source* de mundo real, é desconhecido o número de vulnerabilidades presentes. Logo, Li et al. (2026), para cada projeto, foi capturado: **(i)** #Relatorios, o total de número de avisos produzidos, **(ii)** #Arquivos, o número de arquivos fontes distintos que contém pelo menos um aviso, e **(iii)** #Amostras FPs, o número de falsos positivos identificados pelas ferramentas dentre os avisos. A partir desses títulos, foi estimaa a taxa de detecção de falsos positivos em amostras, do inglês *Sampled* False Discovery Rate (SFDR) como:

$$SFDR = \frac{\#AmostrasFPs}{\#Relatorios}. \tag{2.10}$$

Dentre essas ferramentas, RepoAudit ganhou destaque sendo a ferramenta com melhor *recall* dentre todas as vulnerabilidades de C/C++ avaliadas. Superando CodeQl, SemGrep

Figura 8 – Análise de CWEs (*Memory Leak*, *Use After Free*, *Null Pointer Dereference*)



Fonte: Adaptado para o português para melhor compreensão (LI et al., 2026).

~~e KNighter em todas as CWEs analisadas (LI et al., 2026). Após a avaliação manual dos autores, Li et al. (2026) mostra na figura 8 que a maioria das faltas de detecções são de identificação de fontes e APIs incorretos ou incompletos.~~

~~Para o presente TCC, o trabalho de Li et al. (2026) é essencial para a definição das fraquezas dos métodos implementados até o momento. Com sua análise profunda nas razões de falsos positivos, comparando com métodos tradicionais e baseados em LLMs, Li et al. (2026) revela um padrão no desafio de minimizar falsos positivos, onde tanto métodos tradicionais e baseados em LLMs possuem. Com seus resultados, sabemos direções em quais áreas necessitam mais atenção para a minimização de detecção de erros, além de seu *dataset* essencial para aperfeiçoamento de regras.~~

2.3.3 Trabalho 3: *RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing* (GUO et al., 2025)

O trabalho de Guo et al. (2025), desenvolvido na Universidade Purdue e publicado na 42ª *International Conference on Machine Learning* (ICML 2025), em Vancouver, Canadá, apresenta o RepoAudit, um agente autônomo baseado em LLMs para auditoria de código em nível de repositório, isto é, a revisão de um projeto inteiro com o objetivo de identificar

bugs.

Os autores partem da constatação de que auditar um repositório com LLMs esbarra em três obstáculos: a janela de contexto limitada, as alucinações, que degradam a qualidade dos relatórios, e o alto custo de tempo e de *tokens* ao analisar projetos de larga escala (GUO et al., 2025). Dividir o repositório em pedaços e submetê-los individualmente ao modelo também não resolve o problema, pois muitas falhas são não locais: dependem de valores que atravessam diversas funções, classes e arquivos. Como exemplo, o número de arestas de fluxo de dados do projeto *shadowsocks-libev* ultrapassa um milhão (GUO et al., 2025).

Além disso, diversos tipos de falha são sensíveis ao caminho de execução. Um vazamento de memória (CWE-401), por exemplo, ocorre quando a memória alocada deixa de ser liberada em ao menos um dos caminhos possíveis do programa, e enumerar todos esses caminhos leva à conhecida explosão de caminhos, já que o seu número cresce exponencialmente com o tamanho do código (GUO et al., 2025). Ao analisar a lista CWE Top 25 de 2024, os autores concluem que 19 das 25 categorias (76%) exigem raciocínio global e sensível ao caminho sobre grafos de chamadas, de dependência de dados e de fluxo de controle, enquanto apenas seis podem ser identificadas por análises localizadas sobre a AST (GUO et al., 2025).

Inspirado no comportamento de auditores humanos, que seguem o código na ordem de execução e percorrem apenas os caminhos relevantes para a propriedade investigada, o RepoAudit é organizado em três componentes (GUO et al., 2025):

(i) Iniciador, que identifica os pontos de partida da análise (as fontes), como os valores `NULL` na busca por desreferenciamento de ponteiro nulo. Para isso, utiliza a biblioteca de *parsing tree-sitter* e um conjunto de casadores de padrões escritos uma única vez para cada tipo de falha; os próprios autores apontam a síntese desses casadores por LLMs como um caminho promissor (GUO et al., 2025);

(ii) Explorador, que percorre o repositório sob demanda, analisando uma função por vez. Para cada função, o modelo é instruído a abstrair o programa, mantendo apenas as instruções relevantes, a tratar ponteiros e apelidos (*aliases*) e a coletar os caminhos viáveis, identificando os fatos de fluxo de dados a partir da fonte. Os resultados são guardados em uma memória do agente, que também funciona como *cache* para evitar análises repetidas, e a exploração só avança para funções chamadoras ou chamadas quando o valor investigado escapa da função atual. Um candidato a *bug* é gerado quando a fonte alcança um ponto crítico (*sink*), como a desreferência de um ponteiro, ou, no caso de vazamentos de memória, quando não alcança nenhuma chamada a `free` (GUO et al., 2025);

(iii) Validador, que reduz as alucinações por dois mecanismos: um analisador baseado em *parsing* confirma que cada fato de fluxo de dados respeita a ordem do fluxo de controle antes de ser armazenado, e uma nova consulta ao modelo verifica se as condições de caminho entre funções diferentes não se contradizem, descartando o relatório caso se

contradigam (GUO et al., 2025).

Na avaliação, o RepoAudit foi executado com o modelo Claude 3.5 Sonnet, com temperatura zero e análise limitada a cadeias de no máximo quatro funções, sobre três tipos de falha presentes na lista CWE Top 25: desreferenciamento de ponteiro nulo (CWE-476), vazamento de memória (CWE-401) e uso após liberação (CWE-416). Foram utilizados quinze projetos reais, com média de 251 mil linhas de código, entre eles subsistemas do *kernel* do Linux, Redis, memcached e ImageMagick (GUO et al., 2025).

O agente reproduziu todos os 21 *bugs* relatados em trabalhos anteriores e encontrou 19 novos, totalizando 40 verdadeiros positivos e 11 falsos positivos, uma precisão de 78,43%, com custo médio de 0,44 hora e US$ 2,54 por projeto (GUO et al., 2025). Nos mesmos projetos, a ferramenta industrial Meta Infer relatou 7 verdadeiros positivos e 2 falsos positivos, mas não pôde ser aplicada a parte dos projetos por depender de compilação, enquanto o Amazon CodeGuru relatou 18 falsos positivos e nenhum verdadeiro positivo (GUO et al., 2025).

O estudo de ablação reforça a importância de cada componente: sem a etapa de abstração, o número de verdadeiros positivos cai 47,50% e o de falsos positivos aumenta 181,82%; sem os validadores, os falsos positivos aumentam 245,45%; e sem a memória utilizada como *cache*, o custo fica de três a quatro vezes maior em média (GUO et al., 2025). Aplicado a nove projetos *open-source* ativos, o RepoAudit detectou 185 *bugs* com precisão de 85,71%, dos quais 95 foram confirmados e 79 corrigidos pelos desenvolvedores (GUO et al., 2025).

Como limitações, os autores apontam que, além dos falsos positivos e negativos causados por alucinações, o custo da análise cresce com o número de fontes presentes no repositório e que a ferramenta não é *sound* na análise entre funções, podendo perder falhas que envolvam cadeias de chamadas com mais de quatro funções (GUO et al., 2025). Em uma avaliação independente, Li et al. (2026) confirmam o desempenho do RepoAudit, que obteve o maior *recall* entre as ferramentas avaliadas para C/C++, superando CodeQL, SemGrep e KNighter.

Para o presente trabalho, o RepoAudit é relevante por três motivos. Primeiro, cobre três das cinco CWEs adotadas neste TCC, oferecendo uma referência direta de desempenho para as mesmas classes de falha. Segundo, demonstra empiricamente que etapas de validação reduzem de forma expressiva os falsos positivos gerados por LLMs, o que fundamenta o agente de validação do *pipeline* proposto (Seção 3.2.2). Terceiro, evidencia o principal contraste com esta proposta: o RepoAudit consulta o modelo a cada auditoria, de modo que o custo se repete a cada execução e cresce com o tamanho do projeto, enquanto este trabalho utiliza a LLM apenas para gerar regras, que depois são executadas pelo SemGrep sem custo de inferência.

2.3.4 Síntese comparativa

A análise individual dos três trabalhos relacionados permite, em conjunto, delinear com precisão o estado da arte em Análise Estática Baseado em LLMs e identificar lacunas que este TCC busca preencher. Para facilitar essa análise, a Tabela 2 sintetizam as características técnicas dos trabalhos relacionados entre si e o diferencial introduzido pela proposta deste trabalho.

Tabela 2 – Comparação técnica entre os trabalhos relacionados

Critério	**Yang et al. (2025)**	**Hou et al. (2025)**	~~**Li et al. (2026)**~~ **Guo et al. (2025)**	**Este trabalho**
Foco principal	Geração automatizada de verificadores estáticos a partir de histórico de *patches*.	Geração, otimização e migração multilinguagem de regras de análise estática.	~~Estudo empírico comparativo de detectores baseados em LLMs em escala de projeto.~~ Auditoria autônoma de repositórios inteiros por um agente LLM guiado por fluxo de dados.	Geração de regras automatizada a partir de histórico de *patches* e *datasets*.
Método/ Tecnologia	LLMs + Clang Static Analyser (CSA)	LLMs (*few-shot learning*) + SemGrep	~~Avaliação de múltiplos agentes/geradores LLM frente a análisadores tradicionais.~~ LLM (Claude 3.5 Sonnet) + agente com memória e validadores (*tree-sitter*).	LLMs (*few-shot learning*) + *CWEs datasets* + SemGrep
Métrica Principal de Avaliação	Taxa de validade dos *checkers* e volume de novos *bugs* detectados.	Validade sintática/ funcional e eficácia na transposição de regras entre linguagens.	~~*Recall*, Taxa de Falsos Positivos e custo computacional (tempo/ *tokens*).~~ Precisão, *bugs* verdadeiros detectados e custo (tempo e US$ por projeto).	*Recall*, Taxa de Falsos Positivos e custo computacional (tempo/ *tokens*).
Limitação / Trabalho Futuro	Acoplamento estrito ao ecossistema C/C++ e geração de falsos positivos diante de lógicas de estado complexas.	Alta taxa de falsos positivos e dificuldade em fluxo de dados.	~~Taxas de falso alarme insustentáveis decorrentes de raciocínio de fluxo raso e custo de inferência proibitivos.~~ Custo proporcional ao número de fontes e análise	–

3 Desenvolvimento / Implementação / Experimentos

Este capítulo apresenta a proposta de desenvolvimento deste trabalho, detalhando a arquitetura do sistema, as decisões metodológicas, o planejamento experimental e o cronograma de execução previstos para TCC2. Expandindo como será realizada cada etapa da solução, bem como serão apresentados prazos para cada fase. Em sincronia com a proposta de pesquisa, conforme orientado por Wazlawick (2020), com o foco está na especificação precisa do que será implementado e avaliado.

3.1 Metodologia da pesquisa

Este trabalho utiliza o método *Design Science Research* (DSR), conforme definido Hevner et al. (2004), como método científico orientador. O DSR é um paradigma de pesquisa fundamentalmente focado na resolução de problemas, responsável por produzir quatro tipos de artefatos: constructos, modelos, métodos, instanciações (HEVNER et al., 2004).

O ciclo deste trabalho está estruturado em quatro fases: (1) **construção de *dataset***, organização e identificação de *datasets* públicos e banco de *commits* de bases de código de projetos *open-source*; (2) ***pipeline* de geração de regras**, aplicação de LLMs em um *pipeline* para gerar regras satisfatórias para a ferramenta SemGrep; (3) **avaliação de resultados**, analisar cada relatório para obter taxa de falsos positivos no *dataset* de aprendizado; (4) **aplicação no mundo real**, com o conjunto de regras, será analisado os resultados em projetos *open-source* como Linux Kernel.

3.2 Detalhes da implementação

3.2.1 Visão geral da solução proposta

A solução proposta consiste em uma Extensão Geradora de Regras baseado em LLMs direcionada a Projetos de Larga Escala para a ferramenta de análise estática, SemGrep. O *pipeline* recebe o código-fonte como entrada e utiliza uma LLM para analisar trechos de verificações de segurança e *border-cases* não-tratados. A partir da identificação padrões e potenciais vulnerabilidades, o sistema gera automaticamente regras personalizadas compatíveis com o SemGrep.

Figura 9 – Arquitetura do pipeline proposto

Dataset	→	**Análise**	→	**Geração de Regras**	→	**Otimização**	→	**Ferramenta**
Código-Fonte		Possíveis CWEs		Regras dinâmicas		Fusão de casos		Conjunto de regras

Fonte: Elaborado pelo autor (2026)

3.2.2 Arquitetura do trabalho

Este trabalho é organizado em dois módulos principais independentes e complementares: um *dataset* global unificado de CWEs e uma camada de *pipeline* geradora de regras baseada em LLMs. Capaz de gerar regras a partir de qualquer projeto e analisar os *commits* somando o conhecimento do *dataset*.

1. **Módulo de *Dataset***: A base de conhecimento do sistema é estruturada a partir de do repositório provido por NSA Center for Assured Software (2017). Esta fonte fornece um volume expressivo de amostras de código categorizadas por CWEs, viabilizando o treinamento e a validação do modelo. Para complementar essa base estática e garantir a cobertura de *corner-cases* do mundo real, o módulo também realiza a coleta e integração de históricos de *commits* extraídos de projetos *open-source* ativos. Além da base de NSA Center for Assured Software (2017), será coletado históricos de *commits* de projetos *open-source* ativos, obtendo mais exemplos de *corner-cases*.

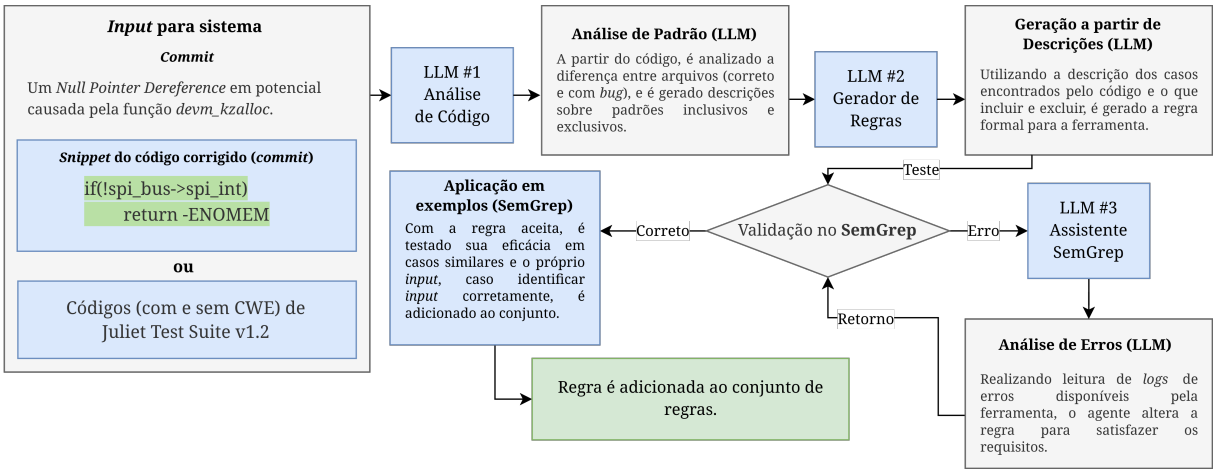
2. **Camada do *Pipeline* (Agentes LLM)**: O fluxo de abstração, síntese e otimização das regras ocorre de maneira iterativa, orquestrado por um conjunto de agentes especializados. Em vez de uma execução linear rígida, o sistema adota um ciclo de retroalimentação visualizado integralmente na Figura 10 e na Figura 11. O funcionamento da camada abstrai-se nas seguintes frentes:

    - **Análise e Abstração (Agente #1)**: Recebe os fragmentos de código (vulneráveis e corrigidos) e sintetiza um relatório destacando os padrões diferenciais e a lógica da falha.
    - **Síntese de Sintaxe (Agente #2)**: Traduz o relatório abstraído para a gramática formal de regras exigidas pela ferramenta SemGrep.
    - **Assistência e Validação (Agente #3)**: Atua no controle de qualidade; caso o motor do SemGrep acuse erros sintáticos durante a validação da regra gerada, este agente interpreta os *logs* de erro e aplica as correções necessárias de forma iterativa. Caso a regra continua rejeitada pela ferramenta depois de três tentativas, será descartada, para evitar consumo infinito de memória e *tokens*.
    - **Otimização Contínua e Consolidação de Regras (Agente #4)**: Após a aprovação e eficácia comprovada, as regras são integradas ao conjunto principal.

Este agente realiza uma varredura periódica para unificar regras que tratam do mesmo CWE, consolidando padrões e maximizando a abrangência de *corner-cases*.

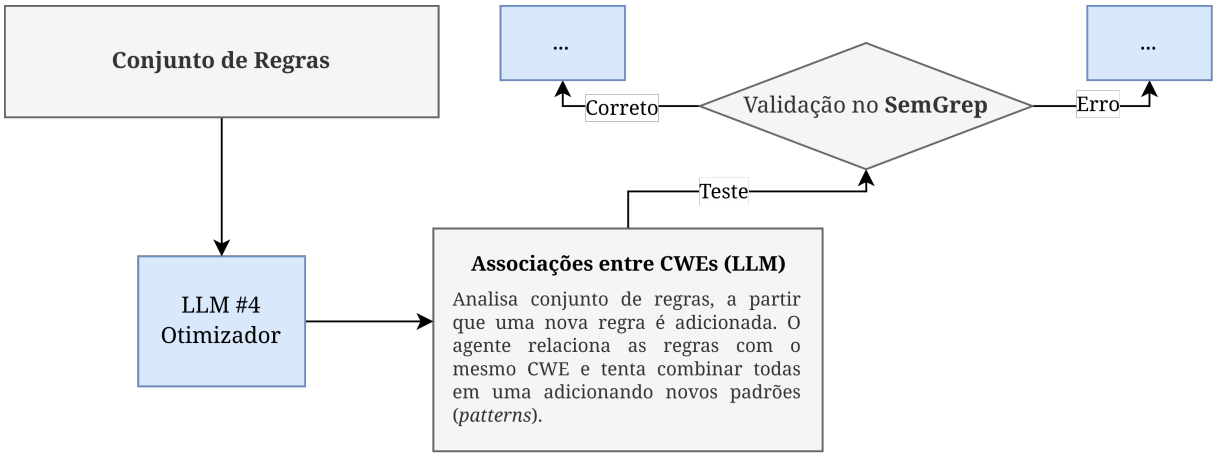
Alinhado aos objetivos específicos (Seção 1.3.2), a arquitetura foi desenhada priorizando a adoção de modelos de linguagem de menor escala (com menos parâmetros) para a orquestração destes agentes. Essa decisão visa viabilizar a execução do *framework* em infraestruturas locais, otimizando o consumo computacional e o gasto de *tokens*.

Figura 10 – *Pipeline* para gerador de regras.



Fonte: Elaborado pelo autor (2026)

Figura 11 – Pós-processamento para otimizar regras e abranger *corner-cases*



Fonte: Elaborado pelo autor (2026)

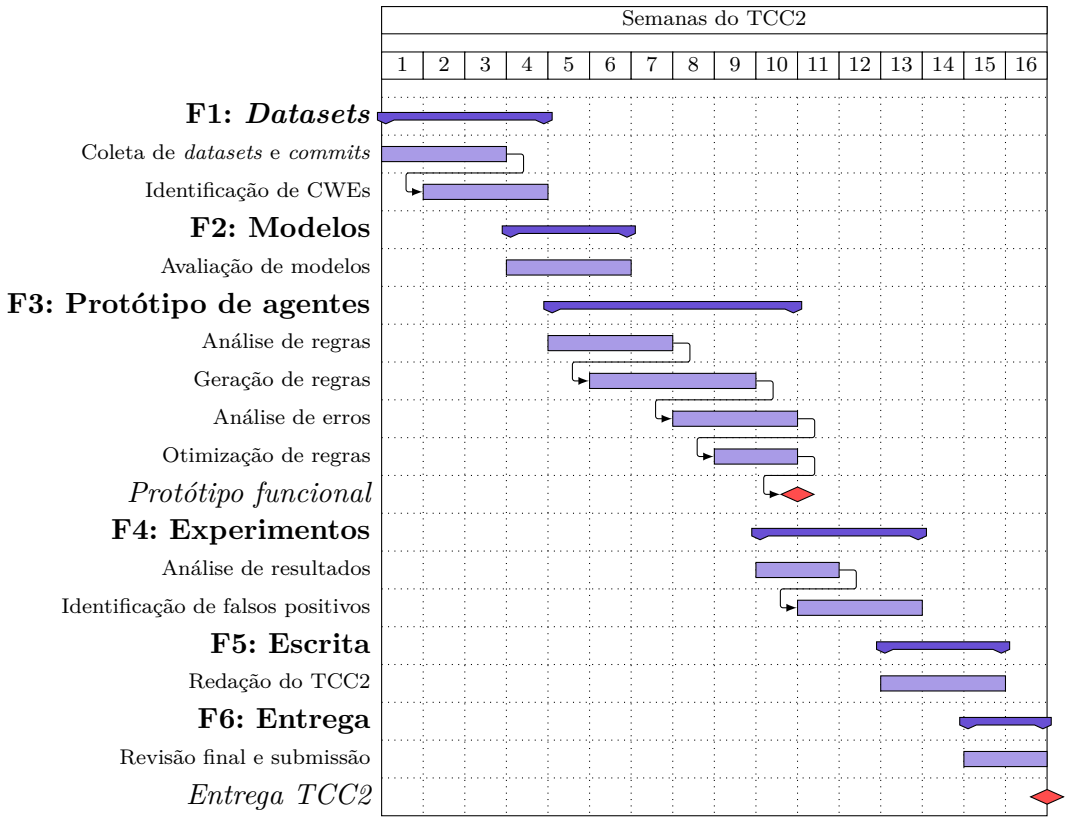
3.3 Experimentos e cenário de testes

3.4 Resultados e análises

3.5 Cronograma de trabalho

O cronograma a seguir está organizado as atividades do TCC2 em 16 semanas, conforme ilustrado no diagrama de Gantt da Figura 12.

Figura 12 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

4 Conclusão

4.1 Retomada do problema e dos objetivos

4.2 Contribuições para a Computação

4.3 Limitações e trabalhos futuros

Referências Bibliográficas

AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2nd. ed. [S.l.]: Addison-Wesley, 2007. ISBN 0-321-48681-1. 12, 20, 21, 22, 23, 24, 38

CVE Program. *About CVE*. 2026. Disponível em: <https://www.cve.org/About/Overview>. Acesso em: 4 jun. 2026. 27, 31

DURUMERIC, Z. et al. The matter of Heartbleed. In: *Proceedings of the 2014 Conference on Internet Measurement Conference (IMC '14)*. Vancouver, Canada: ACM, 2014. p. 475–488. 10

GUO, J. et al. RepoAudit: An autonomous LLM-agent for repository-level code auditing. In: *Proceedings of the 42nd International Conference on Machine Learning*. [s.n.], 2025. Disponível em: <https://openreview.net/pdf?id=TXcifVbFpG>. 9, 40, 45, 46, 47, 48, 49

HEVNER, A. R. et al. Design science in information systems research. *MIS Quarterly*, JSTOR, v. 28, n. 1, p. 75–105, 2004. Disponível em: <https://wise.vub.ac.be/sites/default/files/thesis_info/design_science.pdf>. 52

HOU, Z. et al. Generation, migration and optimization of cross-language static-analysis rules based on large language models. In: *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <https://ieeexplore.ieee.org/document/11173453/>. 9, 10, 11, 12, 13, 14, 16, 17, 24, 25, 40, 42, 43, 44

JIANG, Z. et al. Fact-aligned and template-constrained static analyzer rule enhancement with llms. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2025. 10, 11, 12, 24, 25

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd draft. ed. [s.n.], 2026. Em preparação. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf>. 32, 33, 34, 35, 36

LI, F. et al. *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. 2026. Disponível em: <https://arxiv.org/pdf/2601.19239>. 16, 24, 39, 42, 44, 45, 46, 47, 49

Microsoft Security Response Center. *A proactive approach to more secure code*. 2019. MSRC Blog. Disponível em: <https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>. 10

MITRE Corporation. *CWE - Common Weakness Enumeration*. 2026. <https://cwe.mitre.org/>. Acessado em: 3 de junho de 2026. 27, 28

MITRE Corporation. *CWE-401: Missing Release of Memory after Effective Lifetime*. 2026. <https://cwe.mitre.org/data/definitions/401.html>. Acessado em: 4 de junho de 2026. 28, 29

MITRE Corporation. *CWE-415: Double Free*. 2026. <https://cwe.mitre.org/data/definitions/415.html>. Acessado em: 4 de junho de 2026. 29, 30

MITRE Corporation. *CWE-416: Use After Free*. 2026. <https://cwe.mitre.org/data/definitions/416.html>. Acessado em: 4 de junho de 2026. 29, 30

MITRE Corporation. *CWE-457: Use of Uninitialized Variable*. 2026. <https://cwe.mitre.org/data/definitions/457.html>. Acessado em: 4 de junho de 2026. 30

MITRE Corporation. *CWE-476: NULL Pointer Dereference*. 2026. <https://cwe.mitre.org/data/definitions/476.html>. Acessado em: 4 de junho de 2026. 31

MØLLER, A.; SCHWARTZBACH, M. I. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024. Notas de aula. Disponível em: <https://cs.au.dk/~amoeller/spa/>. Acesso em: 27 abr. 2026. 10, 11

NSA Center for Assured Software. *Juliet C/C++ 1.3 - NIST Software Assurance Reference Dataset (SARD) Test Suite #112*. [S.l.]: National Institute of Standards and Technology (NIST), 2017. <https://samate.nist.gov/SARD/test-suites/112>. Acesso em: 29 maio 2026. 37, 42, 53

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2020. ISBN 978-0134610993. 14, 36, 37

SEACORD, R. C. *Secure Coding in C and C++*. 2nd. ed. Addison-Wesley Professional, 2013. ISBN 978-0321822130. Disponível em: <https://raw.githubusercontent.com/DarkCodeOrg/welcome-to-cybersecurity/master/securecodingincandc++.pdf>. Acesso em: 27 abr. 2026. 26, 27

Software Engineering Institute. *SEI CERT C Coding Standard: Rules for Developing Safe, Reliable, and Secure Systems*. 2016 edition. ed. Carnegie Mellon University, 2016. Acessado em: 2 de novembro de 2026. Disponível em: <https://chenweixiang.github.io/docs/SEI_CERT_C_Coding_Standard_2016_Edition.pdf>. 26, 28

TANG, Y. et al. Grammar-based patches generation for automated program repair. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 2021. p. 1300–1305. Disponível em: <https://aclanthology.org/2021.findings-acl.111/>. 12

The Chromium Projects. *Memory safety*. 2020. Chromium Security. Disponível em: <https://www.chromium.org/Home/chromium-security/memory-safety/>. 10

TUNSTALL, L.; WERRA, L. von; WOLF, T. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc., 2022. ISBN 978-1098103248. 13, 32, 33, 34

VASWANI, A. et al. Attention is all you need. In: *31st Conference on Neural Information Processing Systems (NIPS 2017)*. Long Beach, CA, USA: [s.n.], 2017. 13, 33, 34, 35

WAZLAWICK, R. S. *Metodologia de Pesquisa para Ciência da Computação*. Rio de Janeiro: Elsevier Brasil, 2020. 52

YANG, C. et al. Knighter: Transforming static analysis with llm-synthesized checkers. *arXiv preprint arXiv:2503.09002*, 2025. Disponível em: <https://arxiv.org/abs/2503.09002>. 9, 11, 12, 13, 14, 16, 40, 41, 42, 43, 44, 46